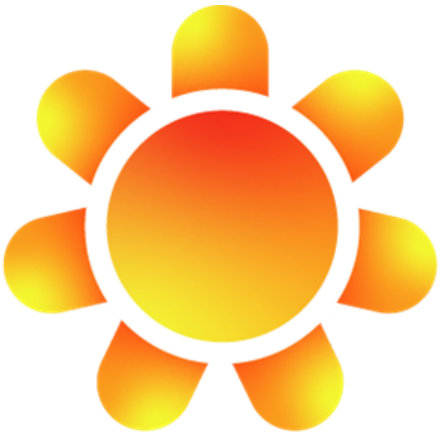


“Seats For All”

Group 2 Report

Abstract

Games have been a simple gateway to socializing since the dawn of time. But over time, the increasing complexity of games has made them more and more challenging to approach, especially for children with Neurodevelopmental Disorders (NDD), who may end up missing out on core experiences. Our aim is to bridge this gap in complexity, and allow kids to return to playing games for the reason they existed in the first place: having fun. Seats for All is a solution that is meant to guide children through the often cumbersome steps of playing a board game, and help them express their creativity and ideas in a way that makes them feel self-sufficient, combining ML detection and Realtime AI technologies.

The Team

Marco Caramazza

Pietro Gianasi

Francesco Torella

Tasks

RESPONSIBLE: Realtime agent implementation, card descriptor implementation, VR setup, audio streaming, game state logic refinements, detection optimization and stabilization, realtime summarization.

CONTRIBUTOR: Prompting, documentation, implementation of UI elements, Azure setup, VR card detection, assisted with animations.

RESPONSIBLE: Curator of the application's overall aesthetics, including stylistic choices regarding interface colors, game fonts, and trims, as well as the creation and development of the assistant, its identity, and appearance.

CONTRIBUTOR: Graphic design of the product identity, Design of the card loading wheel, Design of the card scanning wheel, Choice of application identity colors, Design and creation of the digital assistant ("Seats").

RESPONSIBLE: Card detection model training, board detection model training, dataset acquisition for training, Azure function tools implementation, game state logic setup.

CONTRIBUTOR: Realtime agent implementation, assistant prompting, assistant function tools implementation, Azure setup, documentation, assistant animations.

Contents

1 Executive Summary **5**
2 Requirements **6**
2.1 Stakeholders 6
2.2 Needs 7
2.3 Goals 7
2.4 Constraints 8
2.5 Context 9
2.6 Considerations on MULCA 10
3 State of the art **12**
3.1 Tabletop Games as a Social Interaction mean for neurodivergent children 12
3.2 Use of AR/XR or VR Technologies 13
3.3 Use of LLM Technologies 13
3.4 Why Dixit? 14
3.5 Conclusions 15
4 Solution - UX Design **16**
4.1 General Approach 16
4.1.1 Brief overview of Dixit's rules 16
4.1.2 Major UI decisions 17
App identity 18
Seats appearance 19
Minimalist user interface 20
4.1.3 Major UX decisions 21
An entirely handsfree experience 21
Interacting with Seats 21
A reactive assistant 24
4.2 User Workflow 25
4.3 User Scenarios 27
5 Solution - Implementation **32**
5.1 Hardware Architecture 32
5.2 Software Architecture 33
5.2.1 3D Model - Blender 33
Blockout and polygonal modelling 34
Animation rigging 40
Animation management 41
5.2.2 Artificial Intelligence - Microsoft Azure 43
Realtime - Seats' voice 43
GPT-4o - Seats' eyes 49
Models Coordination 52

5.2.3 Machine Learning Model - Roboflow & Ultralytics 52
Data Acquisition 53
Annotations and Augmentations 55
Training and Integration in Unity 56
5.2.4 Game Engine - Unity 57
Game state logic 57
Meta and XR 60
Inference Engine 62
Animation Integration and Playback in Unity 64
Useful external tools 66
6 Value Proposition 67
6.1 Value for Users and Uniqueness 67
6.2 Why it is a good solution 67
6.3 Competitors and comparison 68
7 Future Work 70
7.1 Critical Reflection 70
7.2 Future developments 70
8 Business Perspectives 73
References 75

1 Executive Summary

SeatsForAll is a VR cooperative experience inspired by and built on Dixit [14], designed to help children engage in a structured social board game without the session constantly depending on an adult to "translate the rules," or the pressures of interacting with others in an unknown environment.

Dixit-like games are powerful because they encourage imagination, perspective-taking, and storytelling, but they also place heavy demands on skills that can be difficult for neurodivergent children: following multi-step instructions, waiting during turns, inferring what is expected from implicit cues, and handling ambiguous outcomes. When these demands accumulate, the child often disengages; not because the game isn't interesting, but because the process becomes cognitively and emotionally expensive. Thus, the child loses a valuable opportunity to connect with others in a meaningful way.

SeatsForAll is designed to respond to these difficulties by turning the session into a guided, self-correcting experience rather than a rule-heavy activity, that allows the child to focus more on their creative abilities, their peers, and most importantly just having fun.

Meet 'Seats', our sunny helper. An embodied, context aware virtual assistant, that can guide the child through the phases of the game, and provide on demand help whenever it is needed.

This is achieved through minimal UI elements, meant to give the child the necessary feedback to feel guided by the application, without it overtaking the social experience of the board game or cluttering the playing environment. In fact, The experience is guided entirely through the child's voice and the visor's cameras, allowing the user to keep playing the game without any extra peripherals.

The implementation was achieved through Unity and the Meta XR packages needed to fully realize the experience, such as the passthrough block required for MR, access to the Meta Quest 3 camera, depth perceptors and raycasters for card detection and tracking, and to place Seats onto the board.

A YOLO detection model was trained through the Roboflow platform, using a dataset of 200+ images. And finally, everything was brought to life through the realtime AI technologies provided by Microsoft Azure, lending Seats a voice through a realtime agent and their creative capabilities through a secondary descriptor.

The value proposition of Seats is a Mixed Reality Dixit experience that is more autonomous, more inclusive and that could be extended to far more complex games in the future, making the tabletop gaming space much more accessible for everyone.

2 Requirements

In this chapter, the main phases of the system requirements elicitation will be presented. The phases in question are the following: identification of project main stakeholders, identification of the needs of those stakeholders, identification of the goals the system should meet to satisfy such needs, identification of the system constraints and finally identification of context in which the system will be adopted. The final section of the chapter will, instead, discuss the use of MULCA (Multi-User Large language Conversational Agent) during the requirements elicitation phases. This tool accompanied the team throughout all the requirements elicitation phases quoted above, as well as the UX design. Therefore the final section will discuss the impact of this system on the design as well as the team feedback on it.

2.1 Stakeholders

The team identified the following potential stakeholders for the system:
Primary Users:

- **Atypical children (Ages 10-14)**: the primary users who the app targets and who will benefit the most from the experience. The aim of the application is to allow them to socialize more easily through table top games.

Secondary users:

- **Typical children:** other children that participate in the game or future users of the applications.
- **Therapists:** the supporters of the atypical child outside the game sessions.

Tertiary Users:

- **Researchers:** interested in the study of how these emergent technologies can combine into something helpful for the future generations.
- **Parents:** can feel reassured their child is making core experience in a stress free environment.
- **Game Designers**: may consider adapting similar models as SeatsForAll as companion apps for their own tabletops, increasing inclusivity.

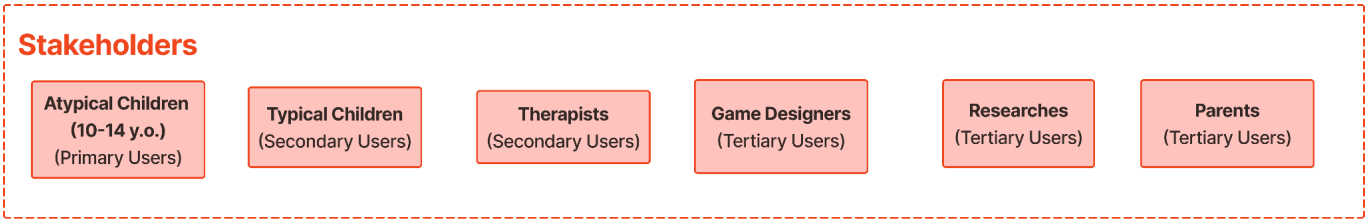


Figure 1: UNG Model - Stakeholders

2.2 Needs

With regard to the stakeholders, the team has identified the following needs:
- **Child's well being:** in the context of the system, these are the needs of our target users, implying:
    - **Social Development:** that includes
        - Sense of Belonging
        - Self esteem and acceptance
        - Coping with stress in social situations
        - Communicate Effectively
- Observe progress in child's development
- Offer continuous support
- Evaluate the effectiveness of new technologies
- Create more accessible games

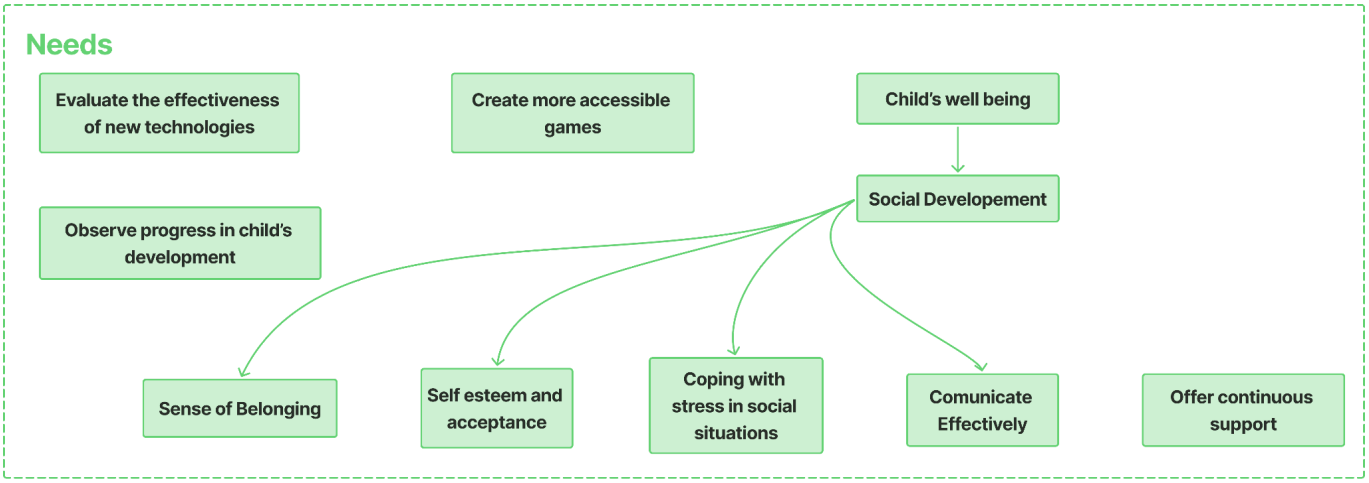


Figure 2: UNG Model - Needs

2.3 Goals

The Goals of the system can be grouped into two macro-categories, "Giving suggestions" and "providing context and rules":

- **Give Suggestions**
    - To not give invasive or over-accurate hints
    - Support with appropriate suggestions
- **To inform on rules and current game status**
    - Give the user a clear overview of the current game status

- Provide clear instructions on which actions can be performed according to the rules

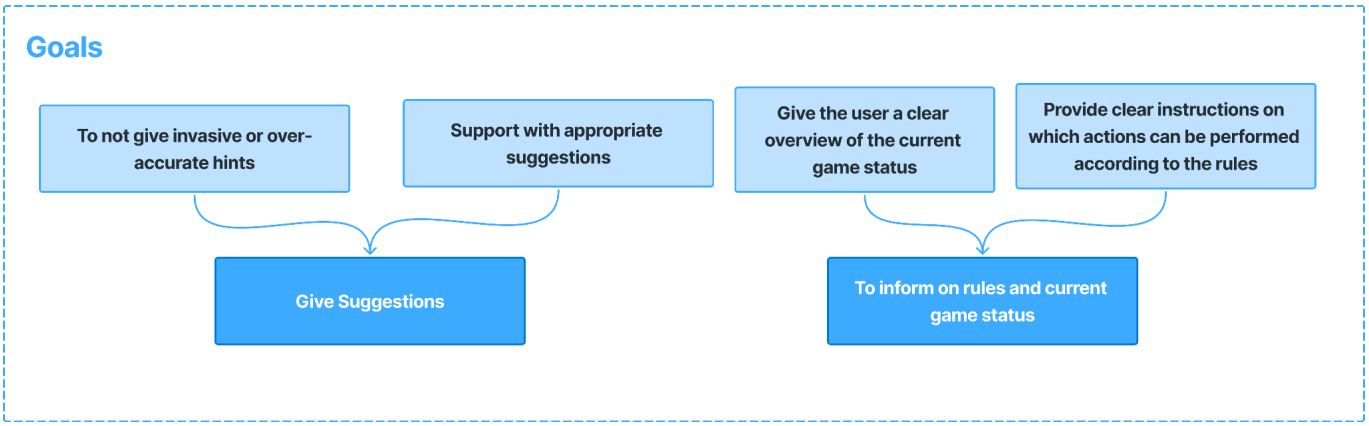


Figure 3: UNG Model - Goals

2.4 Constraints

The system has the following constraints, they can be grouped into categories:

- **Human:** regarding the lack of expertise of the team members.
- **Accessibility related:**
  - The user must be able to read
  - Game complexity hinders certain age groups
- **LLM related:**
  - Response times of the model
  - Limited computational reasoning for inputs and outputs
- **Social:**
  - At least three players are needed
  - Must have at least one typical and one atypical player
- **Technical:**
  - The game will have limited turns
  - The game has a set of rules that must be followed
- **Physical:**
  - Size of the playing area
  - Irregularity of the playing surface
- **Technological:**
  - VR camera resolution is less detailed than eye resolution
  - The potential internet instability

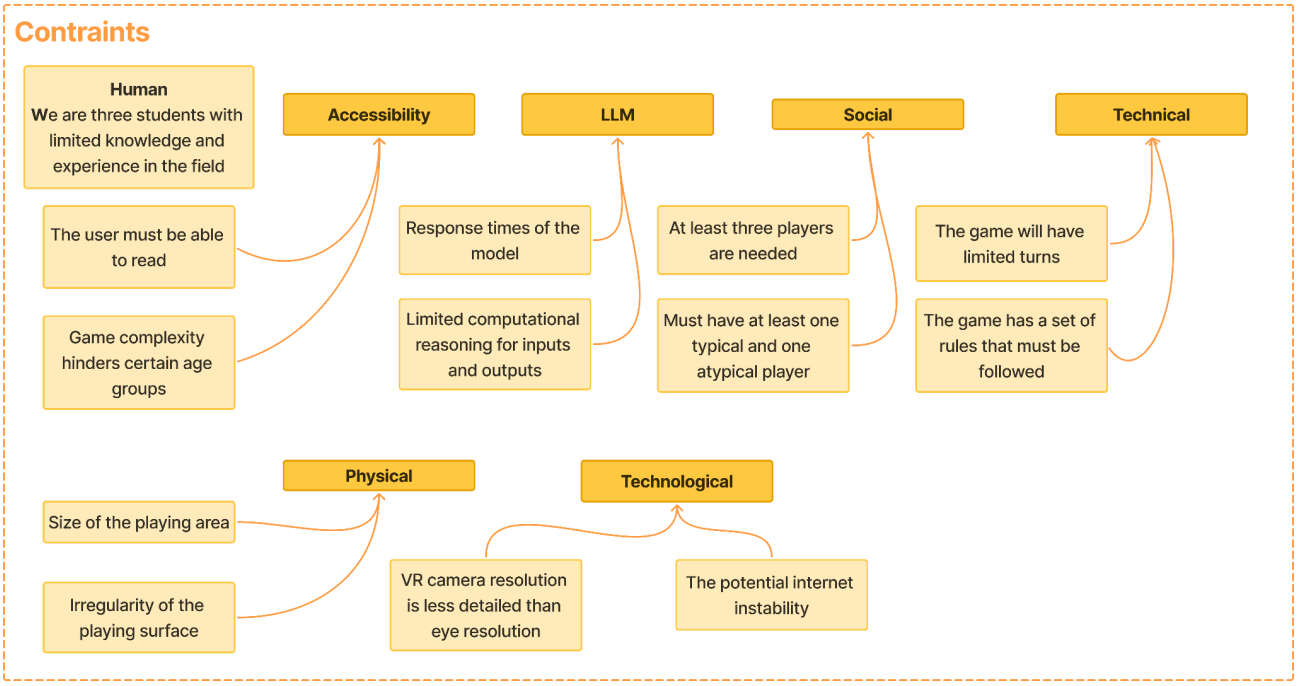


Figure 4: UNG Model - Constraints

2.5 Context

Finally, these are the elements that define the context of the system:

- **Temporal**: during free time or recreational time
- **Social**: at least two players, they can know or not know each other
- **Game context**: all participants want to play and have fun.
- **Technological**: the child plays the game with a visor equipped with a camera
- **Physical**: any sufficiently large space able to fit a table and chairs
- **Audio**: The atypical child must be able to hear the voices of the other children.

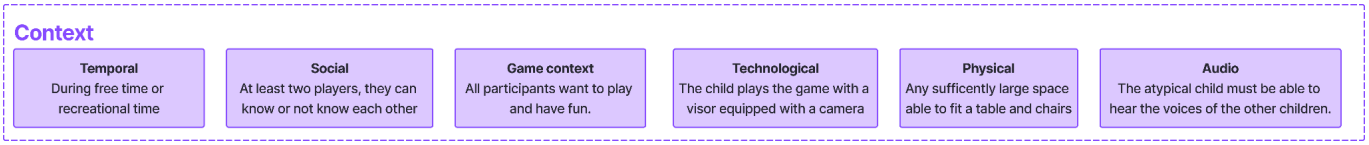


Figure 5: UNG Model - Context

Therefore here is the final UNG schema grouping all the previous categories and showcasing connection between the elements.

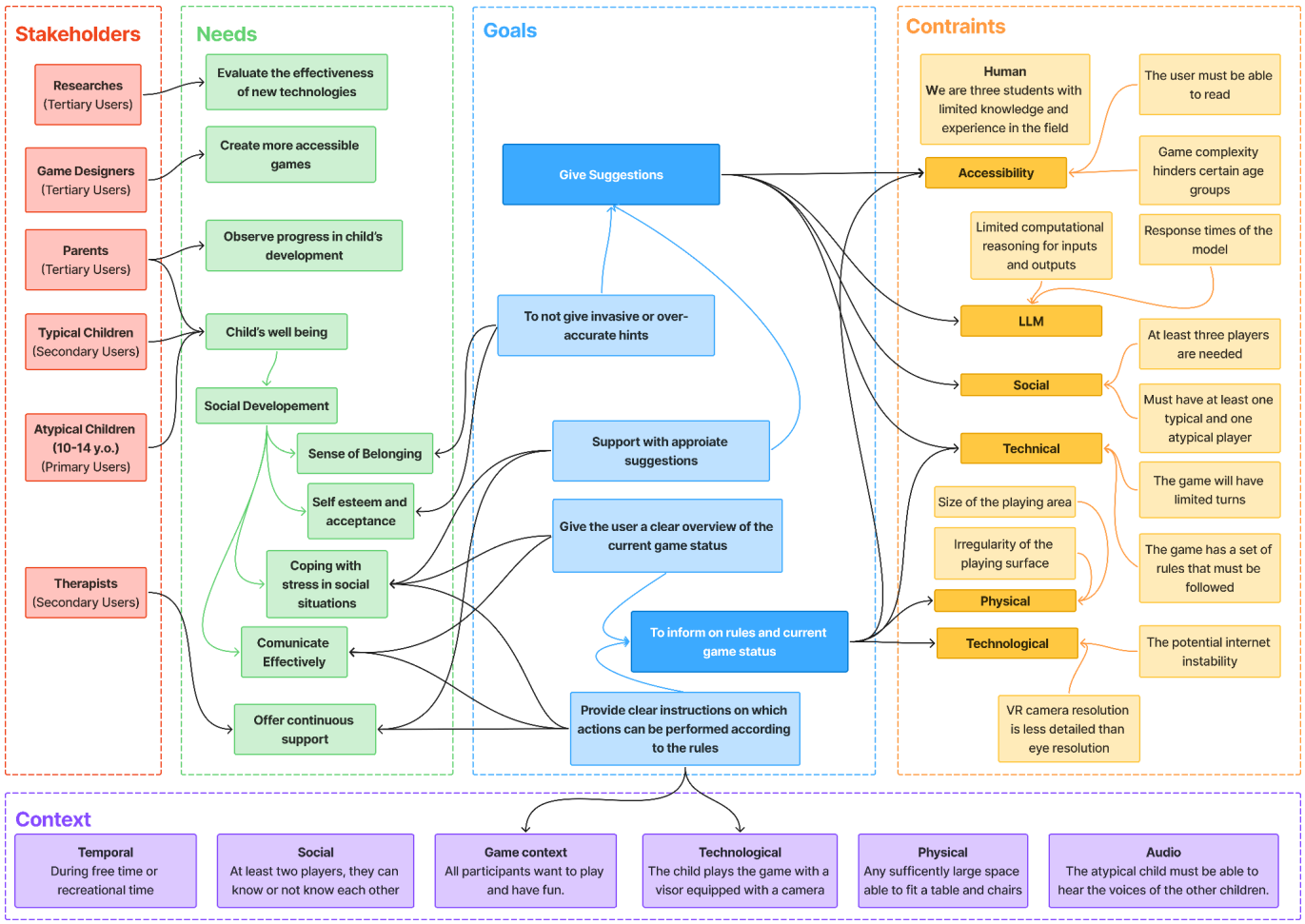


Figure 6: UNG Model

2.6 Considerations on MULCA

A Multi-User Large language Conversational Agent, as anticipated in the introduction, was used through the stages of the requirements elicitation described above. This agent was integrated into Discord (a messaging and voice chat service [13]) as a bot into a server chat. The assistant can reply to chat messages left by users, however it behaves like a conversation listener that interrupts the conversation and leaves a message when appropriate. Therefore the users can brainstorm ideas and share thoughts on their own and sometimes MULCA will leave a message stating something about the current topic of conversation. This means that prompts to MULCA have to be implicit and directed to other members, the team experienced that explicit prompts addressing specifically the agent will often lead to no answer.

The design session took place in the following way: team members reunited on the Discord vocal chat and exchanged thoughts, sometimes prompts to MULCA were left in implicit form as described above waiting for a response. The sessions lasted more or less two hours each.

Here follows a list of observations on the behavior of MULCA

- The team noticed that MULCA actually behaved well when it came to brainstorming; in fact, after typing a set of ideas from the group, MULCA would often reply with other pertinent ideas.
- MULCA seemed to have very short memory, in other words MULCA did not remember the conversation context of past messages and forgot information after a few messages.
- MULCA did not behave well as a knowledge base either: the team had the feeling that the ideas that MULCA had were not supported by a knowledge base.
- MULCA seemed to not propose good ideas to the team after being pushed too much, in fact, these attempts will often lead MULCA to propose ideas out of context..

Even though MULCA did not have an optimal behavior it encouraged a lot of discussion among team members. The fact that team members discussed whether a thing should have been asked or not to MULCA often led to sharing ideas, agreeing or disagreeing on that topic. Finally, MULCA sessions forced all three members to meet (virtually) together at the same time, this meant that any decision on the UX design and requirements elicitation that had been taken had been thoroughly discussed together. This is something that ought not to be taken for granted since students, often having different schedules, may decide to work on the UX design separately leaving a little space for discussion.

3 State of the art

This paragraph presents a short survey on the scientific literature related to the project. Three aspects will be covered: whether table top games can help atypical children to feel more accepted and included, whether technologies such as AR/XR can enhance the interaction even more thanks to their powerful visual stimuli and finally whether LLMs can actually support complex in-game decisions. As suggested by findings in the literature, the many positive results observed were still limited to labs. The general impression, as the following sections will explain, is that most of the studies were conducted only in a lab and in therapeutic context, given that most of the support tools (like therapists or machines) could not be brought outside a lab. Such support tools are therefore limited to specific scenarios and attempts to bring them in "real-life" situations have not been made yet.

3.1 Tabletop Games as a Social Interaction mean for neurodivergent children

The efficacy of tabletop games as means of social interaction and inclusion for atypical children is proved by several studies [10]. Such studies report augmented social interaction, cooperation and communication capabilities. This is particularly due to the presence of turns, roles, and common in-game goals. The category of games that seems to be the most effective as a social interaction tool is role-based narrative games (TTRPGs); this is due to the fact that they stress particularly on roles and communication. Moreover, the game context often allows to train certain social behaviors in a safe and structured way. As a consequence of such positive interactions, children often experience increased self-esteem, sense of belonging, stress reduction and reduced isolation.

Regarding Children with Autism Spectrum Disorder, a survey [2] on several scientific publications underlined how certain games can contribute to emotional regulation, emotional recognition and enhanced eye contact. Unfortunately the games quoted in the survey are specifically designed for therapeutic purposes and are not "real-life" games. Furthermore, the studies quoted in this chapter do not include the use of immersive technologies such as AR/XR but only the potential of games as social interaction enhancers.

3.2 Use of AR/XR or VR Technologies

As several studies suggest [3] [5] [6] [7] [9], the use of AR technologies can function as a bridge between the digital and physical world supporting comprehension of game mechanics, attention and cooperation. This support is offered mostly by the sensorial stimuli that these technologies can provoke and their ability to instantly capture the children's attention. These stimuli also drastically improve game performance; the support of such technologies caused children to feel "safer" and interactions towards therapists were enhanced.

Unfortunately, most of the technologies adopted by the literature cited above required long and expensive setups. In fact, most of these technologies could not be brought into real-life physical environments but were limited to in-lab specific setups. This connects to the problem stated in the introduction related to bringing the support given by the lab into real-life scenarios.

Nevertheless, most of these studies were conducted and designed with a pure therapeutic goal in mind. Specifically, social interactions through table top games were designed in a therapeutic context and not in a "real life" scenario where therapists are absent and children are in everyday situations. This means that atypical children find themselves in a secure environment where mistakes or doubts are tolerated, something that may well not happen in real-life scenarios.

Most of the games used in the studies often targeted children-therapist interactions and they were mostly individual experiences; thus, they lacked the complexity of peer-to-peer interactions caused by real-life tabletop games. Only a few [1] involved more than one child. On top of that, the games played lacked the complexity of real-life games and were, again, specifically designed in the lab. In fact, most of them were reduced to simple motion or object moving; if rules or roles were present, they were often very simple.

Finally, most of these studies only consider AR environments and XR is limited to a few studies [1] [4].

3.3 Use of LLM Technologies

Since AI is still a recent (and also hard to predict) technology, the literature did not explore yet the potential of AI for therapeutic purposes. However, aspects such as board game learning were explored by the literature [8]. The results show that LLMs is very effective in understanding thoroughly in-game mechanics and providing descriptions of them. However, the focus of that study was using AI to decide which game to choose or how to modify a certain game to serve

specific learning needs. It was, consequently, more focused on designing a game than offering live support. However the fact that AI (in particular Chat-GPT4) can understand game mechanics so well is remarkable. In fact, even though there is no evidence yet that AI can serve effectively as an in-game support (even more so in case of children with fragilities), at least AI can understand game rules and mechanics, having the potential of offering live support in the future.

3.4 Why Dixit?

A study published in 2025 [20], aimed at using Dixit cards to help kids with ASD expressing their perception of the disorder. The study concluded that Dixit cards allowed those individuals to convey ideas and emotions that could not be conveyed using a normal social interaction towards a therapist. This is due to the fact that a face-to-face interview would often make the patients feel uncomfortable and highly stressed. Even though this study did not include actual dixit game sessions, it underlined how Dixit cards have the potential to allow ASD kids to express their creativity and imagination.

Another study published in 2024 [21] interviewed hundreds of table top games fans. The study concluded that a surprising amount of people with ASD seemed to enjoy playing table top games. The reason is that such games have strict rules and mechanics that fit well with the psychology of ASD individuals. However, the most important conclusion of this article is that table top games seem to create an environment that facilitates social development for ASD individuals (similar conclusions to section 3.2 but more specific for ASD). Even though this article does not explicitly quote Dixit, it can be deduced, in combination with the previous article, that the game of Dixit *may* work well as a socialization tool and that ASD individuals may find the artistic cards useful to express their ideas and emotions.

Figure 7: Dixit, box and cards

3.5 Conclusions

The project therefore collocates itself in a totally unexplored niche. In fact, the main purpose is to bridge the three results presented in this section attempting to build a system able to visually support ASD children using XR technology and integrate an LLM to offer help during “real-life” game situations. This system will not require difficult or expensive setups and it represents an attempt to bring the support that the child usually receives in the lab into “real-life” scenarios. If this system proves itself to be useful during such situations, the child will benefit from enhanced social interactions with neurotypical peers and the support will be continuous in time and not limited only to in-lab sessions.

4 Solution - UX Design

4.1 General Approach

The final purpose of this project is to build a system able to support the user during a game of Dixit. This system will be composed of a VR headset and an LLM assistant. The headset will be responsible for the immersion into the MR environment, such an environment will display the visual stimuli and feedback that the child will need as support. On the other hand, the LLM assistant should be able to actively talk and conversate with the child. Its purpose will be to support the child with in-game decisions using their voice.
This section wants to highlight our design choices both UX and UI wise, and what drove those decisions in the first place.

4.1.1 Brief overview of Dixit's rules

First, a brief presentation of the rules of Dixit is useful to understanding the application. The purpose is to allow the reader to better grasp the core decision of the UX design. In fact, such decisions are mostly dictated by the rules of dixit, therefore a brief description of the rules is necessary.

The package is composed of a scoring board, 80 total cards, voting tokens and game markers. Each card is different and they feature an artistic image. The game is played in rounds. Before the first round each player is given a total of 6 cards each. In each round, one player takes on the role of the **Narrator**, then:

1. **The Clue:** The Narrator chooses one card from their hand and, without showing it, gives a clue. This clue can be anything: a word, a sentence, a song title, or even a sound. In order to simplify the work, in the project we assumed the clue to be only a word; even though the assistant could easily support more complex clues. In the rest of the document this word will be referred to as *Narrator's Word*.
2. **The Matching:** Every other player, which in the project are referred to as *Guessers,* chooses a card from their own hand that best matches the Narrator's clue and gives it to them face down.

In this document, the Clue and Matching phases are grouped into one single phase that in this project is referred to as the *Card choosing phase.*

3. **The Vote:** The Narrator shuffles all the chosen cards (including their own) and lays them out. The board features numbered slots for the cards and the Narrator should place each card into these slots in a random way. Guessers then vote on which card they think belongs to the Narrator (they cannot vote their own card). During this phase the narrator does nothing. To vote, each player is given a voting token, this token features a scrolling wheel that allows them to select a number. In order to vote each guesser should select the number of the card they want to vote for and then place the token on that card. To prevent them from cheating by changing the number at the very last second, each player should place their token at the same time.

The voting phase will be referred to as *Narrator's Card Guessing phase.* Lastly points are assigned at the end of each round according to the following rules:

- If *everyone* or *no one* finds the correct card, the Narrator gets 0 points, and every Guesser gets 2.
- In any other case, the Narrator and those who guessed correctly get 3 points.
- Bonus: Guessers (and not the narrator) earn 1 bonus point for every vote their own card received.

The assignment of points will be referred to as *Points Computation Phase*. Once the round is over, the narrator role is passed to the next player and each player draws another card. The first player to reach 30 points is the winner, in case more than one player reaches 30 points the victory is split.

The scoring board, which will be simply referred to as the *board,* contains 30 cells. To allow players to keep their scores, each player is given a marker featuring the shape of a rabbit. This marker will be simply referred to as the *rabbit*. Each rabbit has a distinct color to be distinguished and each player chooses their color before the match can start. Each token has also a distinct color that should match the rabbit one.

4.1.2 Major UI decisions

The UI elements of our application, while few by design as it will be described in the following subsection, all carry the intent of designing a user-friendly, low stimulation, and memorable experience.

App identity

The app's logo was a unanimous choice by the team to highlight a graphic detail that we were certain would be fundamental for its visual appeal and the strength of the "Seats for all" identity.

The choice was a result of the project's title itself: it needed a visual element that connected concepts of **well-being, inclusion, balance, and equality**, while still being lively and fun. The element, beyond connecting with the digital assistant's identity, recalls the **visual image of a table surrounded by seven chairs**, emphasizing the importance of company and inclusion.

The **rounded geometric shapes**. are a part of a design strategy aimed at creating an environment **free of visual aggression**.

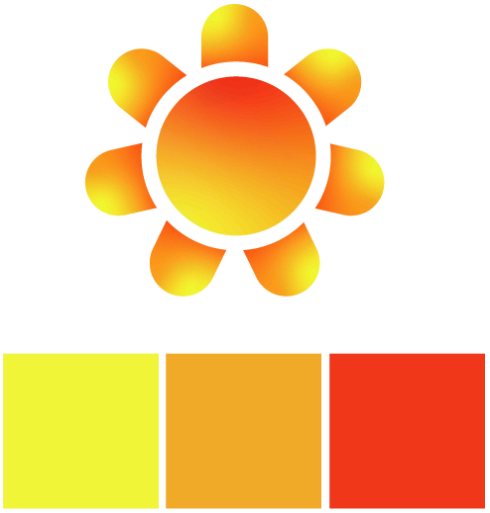


Figure 8: App Logo and color palette

The choice of the color palette was not accidental; the preference for a gradient from **fiery red to bright yellow** aims to evoke a sensation of pleasantness and warmth in the user, as they tend to associate it with concepts like the sun or light.

The logo was integrated in the application as a simple loading wheel, meant to give active feedback to the user while the application went through its card scanning phase.

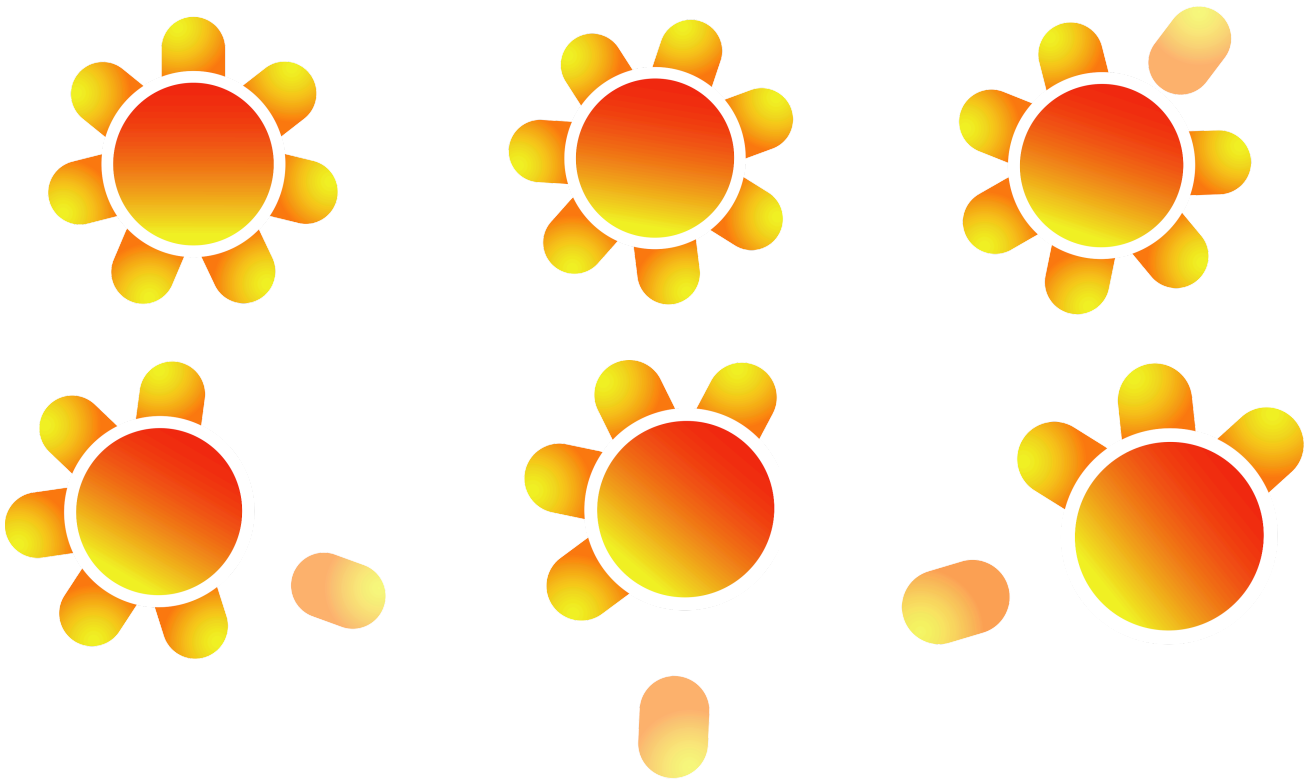


Figure 9: Loading wheel sequence

The movement of the loading wheel was animated using **Adobe Premiere Pro**. For the animation, we opted for a constant rotation around its center with a progressive detaching and subsequent reattaching of the petals, recalling once again the idea of chairs being pulled from and into a playing table.

The result was consistent with the previously mentioned visual identity: a **gentle flow** that creates a visually pleasing animation without overstimulating the user.

Seats appearance

Visual readability was the cornerstone in developing the digital assistant, conceived as a real-time interactive element within an MR environment. For the 3D model, the primary goal was to define an aesthetic capable of instilling tranquility while also recalling the established app identity. Seats should not feel like an overseer of the gaming experience, but rather another friend at the table, welcoming the user into an environment where they can feel cherished by their friends and the playing environment.

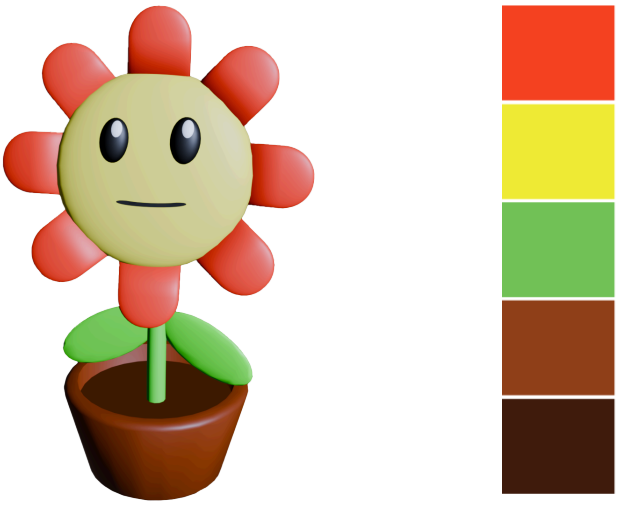


Figure 10: Seats and their color palette

Hence, we opted for a cartoonish model characterized by smooth geometries and a clear separation of functional elements. A simple flower, with forms that are intentionally exaggerated and reduced to their essential components, ensuring that the most expressive features such as the head, eyes, and mouth remain immediately recognizable and communicate Seats' status and intent immediately, minimizing ambiguity and cognitive load for the user.

The color palette was selected to fully respect the character's toon-like nature. The vibrant red of the petals, the vivid yellow of the corolla, and the brilliant green of the stem and leaves immediately capture the user's attention from the moment of Seats appearance, and help leave an impactful memory of the experience.

Minimalist user interface

In terms of presentation, as mentioned in the beginning our application intentionally adopts a minimalist UI language: the experience is anchored by the embodied 3D assistant and a single-card tracking marker that joins the loading wheel with a small set of short text fields meant to aid the user during the experience.

This choice is motivated by the constraints of our environment setting, where the user's attention must not be entirely drawn away from the physical environment, including the other players; adding dense interface elements would have increased visual clutter and impeded the user's primary needs. Moreover, since our target users may be particularly sensitive to cognitive overload, the reduced UI surface helps keep interactions to what feels necessary, ensuring that the MR augmentation supports the activity rather than taking away from it.

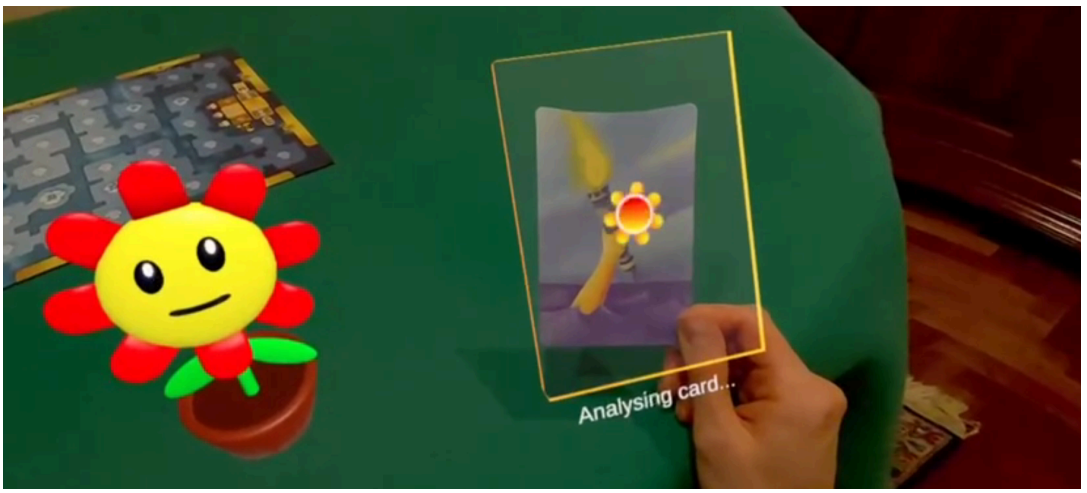


Figure 11: A snapshot of the SeatsForAll UI

4.1.3 Major UX decisions

An entirely handsfree experience

SeatsForAll had to be designed as a fully peripheral-free application due to the nature of the experience we were creating: a player navigates the interactions with the assistant through their voice, while the headset's gaze is used to scan with the physical cards simply by looking at them. The need for controllers or specific gestures was completely removed, allowing the user's hands to remain available for what matters in the moment: playing and communicating freely with others. By reducing interaction to speech and natural head orientation, the system preserves its unobtrusive nature.

Interacting with Seats

Several choices went into designing Seats' identity to be fun, extremely reliable but also discreet. Seats was configured with a child-appropriate voice identity, using the "ballad" Azure voice for a gentle and upbeat delivery that fits both the age range and the creative tone of the game. Voice Activity Detection was specifically calibrated to cope with a noisy, social play space: a strict noise threshold reduces accidental triggers from background chatter, prefix padding preserves the beginning of an utterance once speech is detected, and an extended silence window avoids cutting off users who speak with a slower cadence or pause to think mid-sentence. Together, these parameters aim to make the assistant feel patient.

```
string instructions = EscapeForJson(introductions + gameRules + environment +
responseStyle + functionTools + procedures);;
   string payload = $@"
```

```
    {{
        ""type"": ""session.update"",
        ""session"": {{
            ""instructions"": ""{instructions}"",
            {toolSchemas},
            ""input_audio_format"": ""pcm16"",
            ""voice"": ""ballad"",
            ""input_audio_transcription"": {{
                ""model"": ""whisper-1""
            }},
            ""turn_detection"": {{
                ""type"": ""server_vad"",
                ""threshold"": 0.85,
                ""prefix_padding_ms"": 650,
                ""silence_duration_ms"": 800,
                ""create_response"": false
            }}
        }}
    }}";

    socketClient.SendEvent(payload);
}
```

Listing 1: Seats' identity is configured

The most defining UX decision is that the assistant does not automatically respond every time it detects speech, as it would have created an environment where it is impossible to communicate with other players. Instead, by setting `create_response` to false, the system shifts from a conventional always-responding assistant to a wake-word mediated interaction. Where the system finds innovation, in comparison to other digital assistants such as Siri [34], is by taking advantage of the underlying model's continuous listening to capture the wake-word at any point in the speech.

The application tries to capture when the user says Seats' name, and only respond on that occurrence, reinforcing the user's feeling that they are interacting with another person at the table.

Once the wake-word is heard, Seats opens an interaction window to keep the conversation going without needing to have their name brought up again. The window remains active if the assistant asks a question, during the setup of the game where guidance is more frequent, or if the assistant's reply is short enough to be interpreted as just an acknowledgment; otherwise, the window closes after a brief timeout unless the user speaks again. This design frames Seats as an

on-demand companion that can be addressed instantly, yet stays out of the way when players are focused on each other and the cards.

```
string trimmed = assistantText.Trim();

   if (trimmed.EndsWith("?") || trimmed.EndsWith("¿"))
       return true;

   if (phaseController != null)
   {
       var phase = phaseController.CurrentPhase;
       if (phase == DixitGamePhase.Boot ||
           phase == DixitGamePhase.Setup)
       {
           return true;
       }
   }

   int wordCount = trimmed.Split(new[] { ' ' },
StringSplitOptions.RemoveEmptyEntries).Length;

   if (wordCount <= 8)
   {
       return true;
   }
   return false;
```

Listing 2: Heuristics for the interaction window

While this approach has its drawbacks due to the unreliability of Speech To Text (STT) with syntactic parsing, it was still pursued for **cost and latency optimizations** over semantic parsing, and after multiple testing phases we are fairly confident the system should be able to detect when Seats is addressed.

```
private bool IsAddressedToAssistant(string transcript)
{ //. . .
string[] wakePatterns = new[] {
   "seats", "seat", "sits", "sit",
   "hey seats", "hey seat", "ok seats",
   "seets", "seetz", "seits", "siets",
   "ceets", "cits", "ceats",
   "zeats", "zeets", "zits",
   "sizz", "sizzy", "sitz", "seez", "sez",
   "seas", "sees", "seess",
```

```
    "seed", "seeds", "sed", "sids", "sid",
    "sietz", "seets", "eets",
    "cedes", "seeds", "ceased",
    "si, se", "si, c'è", "setsu"
};
//. . .
}
```

Listing 3: 'Seats' used as a wake-word

A reactive assistant

Seats is designed to behave as a **reactive** companion, using animation to communicate that it is present, attentive, and engaged with the ongoing play. This is a careful consideration in Mixed Reality, where virtual elements can compete with the physical tabletop for attention: a static agent risks fading into the background, while an overly expressive one risks becoming distracting. For this reason, we adopt a deliberately small set of animations that are easy to interpret at a glance.

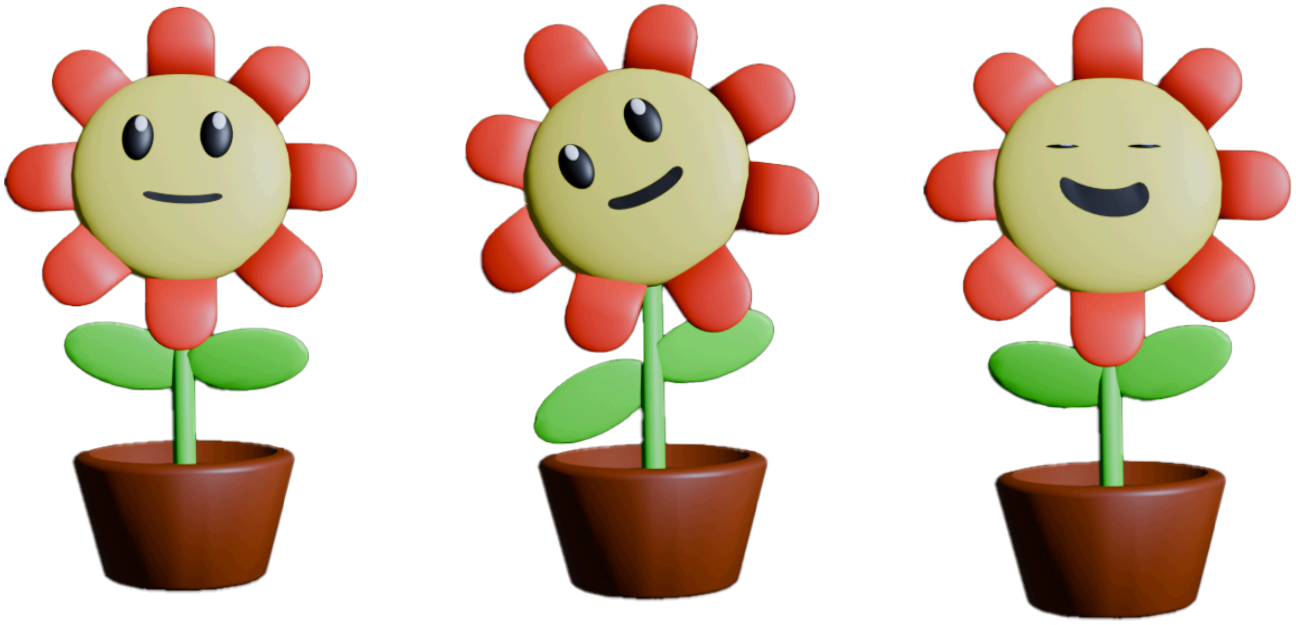


Figure 12: Idle, Talking and Cheering animation

Idle is the default animation, maintaining a subtle sense of aliveness when no direct interaction is occurring and helping Seats feel like a presence in the environment rather than another piece of

the tabletop.
Talking provides immediate visual confirmation that the assistant is actively responding to the user's voice, creating a more inviting and natural conversational environment.
Cheering is used as a rewarding animation once points are calculated, always expressing a positive reaction that supports motivation and maintains a playful attitude even when outcomes are not what the child would have preferred. We put a lot of emphasis on this aspect since children in our target age group are strongly influenced by the elements of their environment and often imitate expressions and behaviors they see around them [22]. Showing a joyful laugh even when the game is not going as expected can improve their perception of the playing experience without discouraging them or triggering negative feelings such as frustration or sadness.

The animations are designed to remain smooth and predictable. Transitions between states are intentionally consistent, avoiding abrupt shifts that could disrupt immersion or produce discomfort, especially for users who may be sensitive to sudden movement or excessive visual stimulation. By limiting expressivity to simple gestures and reserving higher-energy animation for meaningful moments, the assistant remains lively without becoming visually demanding.

4.2 User Workflow

The user workflow is divided into 4 stages:

- **System boot**: in this phase the user is asked to provide the assistant with the necessary information to set up the game. This information includes how many players are playing and which is the initial role of the user (narrator or guesser).
- **Card selection**: if the user is a narrator, they have to choose a card and say a word aloud; if, instead, the user is a guesser, they have to listen to the narrator word and choose a card that may fit the word. In the first scenarios the assistant is meant to offer support in choosing both the word and the card. While in the second scenario, the user may clarify the meaning of the narrator word and suggest which card to play. Note that suggestions on the card are possible only if the system has scanned the cards.
- **Narrator Card Guessing**: this phase begins after every player has chosen a card. The narrator has to collect and shuffle the guessers' cards and then display them on the appropriate spots near the board. If the child is a narrator and is unsure on what to do the assistant will tell them to perform such operations; afterwards the child has no to nothing. On the contrary, if the child is a guesser, they will have to guess which is the card of the narrator. Again, the assistant will provide support by scanning the cards and checking whether they connect or not to the narrator word. Once the child has made this decision the assistant will tell, if needed, to place the voting token on top of the chosen card.

- **Point Computation**: Here each player has to compute their score. Since the scoring mechanics can be tricky due to the volume of rules to remember the assistant will support this phase if directly asked to.

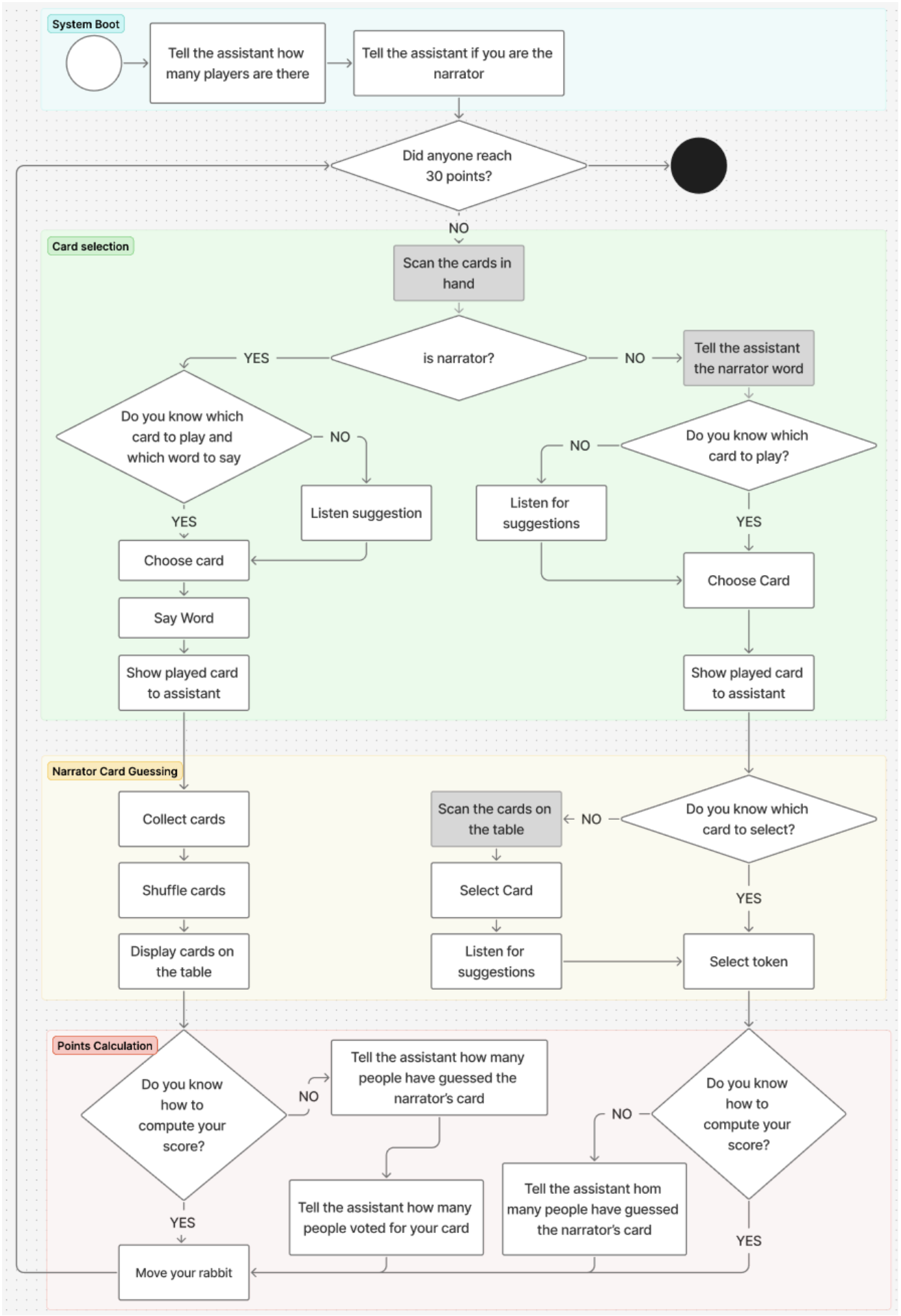

System Boot
Tell the assistant how many players are there
Tell the assistant if you are the narrator
Did anyone reach 30 points?
NO
Card selection
Scan the cards in hand
is narrator?
YES
NO
Tell the assistant the narrator word
Do you know which card to play and which word to say
NO
Listen suggestion
Do you know which card to play?
NO
Listen for suggestions
YES
Choose card
Say Word
Show played card to assistant
YES
Choose Card
Show played card to assistant
Narrator Card Guessing
Collect cards
Shuffle cards
Display cards on the table
Scan the cards on the table
NO
Do you know which card to select?
Select Card
Listen for suggestions
YES
Select token
Points Calculation
Do you know how to compute your score?
NO
Tell the assistant how many people have guessed the narrator's card
Tell the assistant how many people voted for your card
YES
Move your rabbit
NO
Do you know how to compute your score?
Tell the assistant hom many people have guessed the narrator's card
YES

Figure 13: User workflow

4.3 User Scenarios

Title	**Game Setup**
User Profile	Luca, a 13-year-old boy with ASD (Autism Spectrum Disorder), wants to join a game of Dixit with his neurotypical friends. Luca does not know the rules well but he is confident that the system will help him.
User Goal	Luca wants to learn the game by playing it, seeking the opportunity to socialize with his peers and have fun independently
Context	The kids just reunited around the table and they are unpacking the game objects such as the board or the cards. The game is about to start.
Tasks:	Luca puts on the headset and opens the SeatsForAll application. He scans Dixit's game board, Seats appears next to it and welcomes him. Seats asks him for the necessary information to set up the game, specifically: whether Luca is the narrator, how many players there are and his initial hand of cards. Once he provides the information the game can begin.

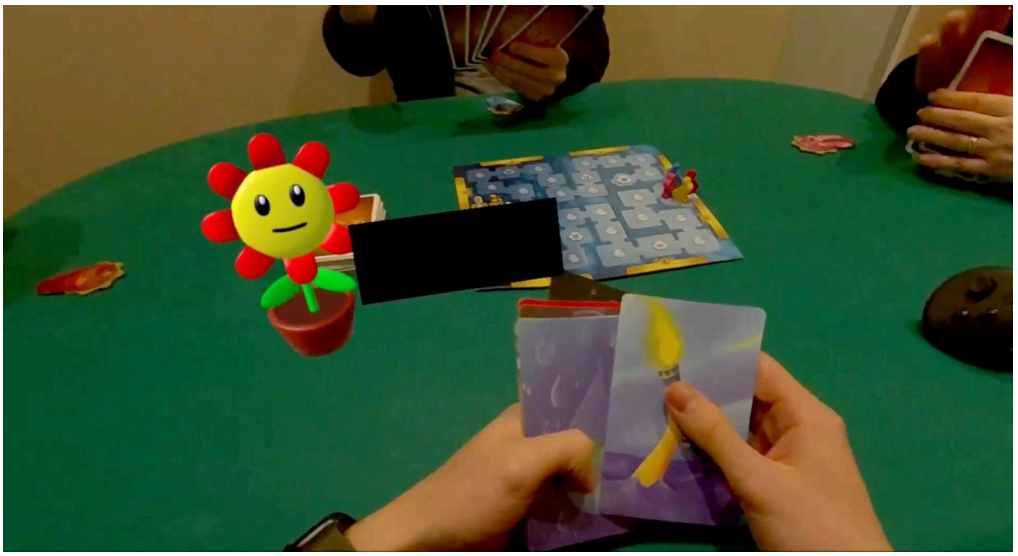


Figure 14: Seats appears next to the board

Title	**Narrator**
User Profile	Marta, a 14-year-old girl affected by ASD (Autism Spectrum Disorder), wants to join a game of Dixit with her neurotypical friends. Marta has played Dixit a couple of times but she does not remember the rules very well.
User Goal	Marta wants to have fun with her friends and have an opportunity to strengthen her bond with them.
Context	At the gaming table, Marta is playing dixit, now it is her turn to be the narrator. She has already scanned her cards.
Tasks:	Marta now must choose which of her cards to play. Feeling indecisive about which clue to associate with one of her cards, Marta asks Seats for help. She takes one card from her hand and frames it with the headset's camera so it can be scanned. The headset highlights the card Marta has picked with an orange outline, indicating the analysis has started with a text label under the outline After a few moments of reflection, the assistant suggests possible clues that could vaguely relate to the chosen card. These suggestions are also displayed as text labels near the card. Marta isn't satisfied with the clues, so she asks the assistant to suggest more words on the same card. Seats gives three brand new suggestions, and Marta finds one that is to her liking. She plays the card and speaks the word aloud, allowing the turn to begin. Finally Marta tells Seats which card she has played.

Figure 15: The clues for the card appear next to the outline

Title	**Guesser**
User Profile	Enea, a 12-year-old boy with ASD (Autism Spectrum Disorder), wants to play a game of Dixit with his neurotypical friends. Enea has played several games of dixit and he knows the rules, however he often struggles with associating cards.
User Goal	Enea would like to play the game without feeling stressed and under pressure.
Context	During the current turn, Enea is a guesser; Enea has already scanned his cards
Tasks	After thinking it over, the Narrator says the word. At this point, each player looks at their own cards to decide which one best matches the word. Enea, however, is struggling and can't choose between his cards. Consequently, Enea calls upon Seats and asks them for help deciding. Seats hasn't heard the narrator word from Enea, so they let him know they'll be ready to help as soon as they are told Enea tells Seats the word and the assistant, having already scanned the cards in his hand, provides three possible cards they could play. Now Enea feels less pressured and he is ready to make his choice.

Title	**Guesser tries to guess the narrator's card**
User Profile	Ludovica, a 13-year-old girl with ASD (Autism Spectrum Disorder), wants to join a game of Dixit with her neurotypical friends. Ludovica knows the rules, but she often finds it difficult to associate the narrator's word to cards.
User Goal	Ludovica would like to have fun playing without feeling stressed and pressured.
Context	At the gaming table, Ludovica is playing Dixit, where players must personally vote on which card belongs to the narrator. Ludovica does not know which card could have been the narrator's card, she feels stressed and pressured.
Tasks	Ludovica tells Seats she can't find any connections between the narrator word and the card in front of her. Seats asks her to scan the card and she frames it with the headset. Seats rapidly responds to Ludovica, explaining possible connections between the card and the narrator's word. Ludovica still isn't so convinced, so she asks Seats for help with a different card and they repeat the procedure. The second time around Ludovica is much more confident of the reasoning, so she decided to vote for that card to be the narrator card.

Figure 16: Seats waits for the card to be scanned

Title	**Points computation**
User Profile	Claudio, a 11-year-old boy with ASD (Autism Spectrum Disorder), wants to join a game of Dixit with his neurotypical friends. Claudio has played the game a couple of

times but it was a long time ago and he does not quite remember all the rules.

User Goal Claudio noticed that his friends play the game often and he wants to remember the rules in order to play and socialize with them.

Context Having reached the end of the round, Claudio does not quite know how many points he scored since he does not remember the rules very well.

Tasks Claudio asks Seats for help in computing the points, who starts asking Claudio a series of simple questions: Was Claudio the narrator? How many players chose his card? Claudio responds that he was the narrator this round, and that some players guessed his cards but not all of them. Seats calculates the score and tells it to Claudio, reminding him to move his game piece forward the appropriate number of spaces. Claudio feels relieved he was reminded of the rules, and moves his token forward on the board.

Figure 17: Seats cheers after computing the points

5 Solution - Implementation

5.1 Hardware Architecture

Meta Quest 3 is our primary platform for the application. Meta's recent developments allowing third-party developers to access the camera's feed through the passthrough API, enabled unprecedented possibilities in the realm of real time VR detection, object tracking, and Mixed Reality applications.

The experience is entirely peripheral free, and the application only requires access to the device's microphone for capturing user speech, and the device's cameras and for the detection of cards. The tracking is made more precise utilizing the visor's integrated depth sensors (In the standard model of the Meta Quest 3) or the Infra-Red ray casters (In the S model).

Figure 18: Meta Quest 3 and 3S

- **Passthrough API:** Raw camera feed from the frontal camera of the device is analyzed by ML detection models every 0.15 seconds, one is trained on recognizing the board of the game and placing Seats near it, the other is trained on Dixit card with objective of detecting and tracking them when the child asks for assistance.

- **Depth Perceptors:** The sensors on the device assist the single card tracking. Using the known measures of a Dixit card (80 x 120 mm), and the perceived distance in world space

through the sensors, we are able to compute an approximate shape of the card. Using these measures and the detection box of the ML model, we can precisely place a 3D marker over the card if it is within 0.5 to 3 meters from the sensor.

- **RGB Color Cameras:** The availability of an RGB passthrough feed on Meta Quest 3, enabled the acquisition of Dixit cards in full colour directly from the device's frontal cameras. This is particularly relevant for LLM analysis, as colour information contributes substantially to the accuracy of the generated descriptions.

In earlier Meta Quest generations, the inaccessibility or low quality of these features, would have made the implementation and execution of the application nearly impossible utilizing our design.

5.2 Software Architecture

While the solution would have been impossible without the aforementioned hardware at its core, what gives it life is its software implementation.

Many different softwares were involved in the development of the application, each building on top of one another to deliver the complete experience.

5.2.1 3D Model - Blender

In the early stages of development, the team unanimously concluded that creating a digital avatar to embody the software's artificial intelligence would make the experience more comfortable for our target users, and give the assistant and the application a more personal identity enhancing the Mixed Reality environment. An embodied assistant is a greater source of comfort for the child, especially in a potentially stressful environment.

Therefore, the creation of a 3D model was a crucial step in the app's development and Blender [35] was used to create the Seats's model and the rig for its animations.

Figure 19: Blender logo

Blockout and polygonal modelling

The first step was to create the model using the software. The process was carefully curated to maintain an organic silhouette; the targeted use of surface subdivision allowed us to translate the softness of the 2D concept into reassuring, edge-free 3D volumes.

The first element modeled was the pot. Starting from a cylinder positioned with the bottom face downwards and moved above the axis origin, we applied the Global Scale command. After selecting all vertices defining the top face, we scaled it to be larger than the bottom face to create the tapered shape.

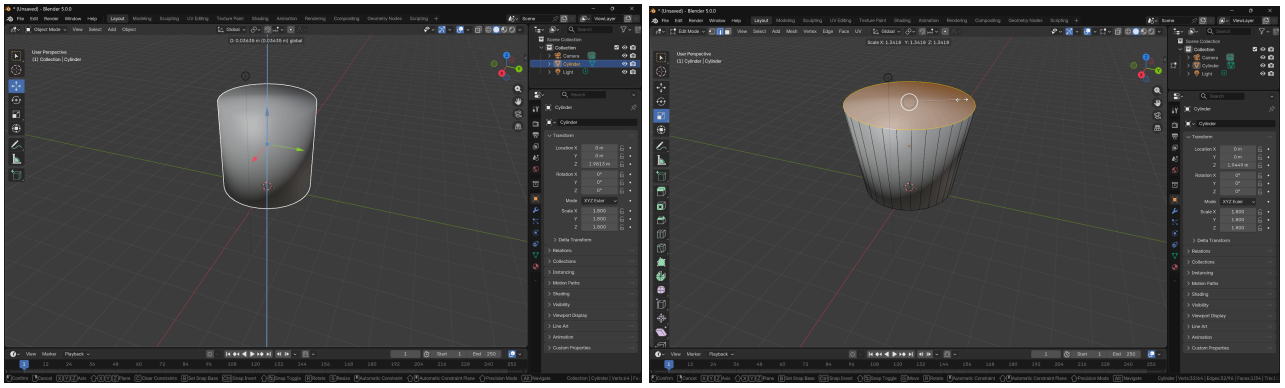


Figure 20: Pot volume definition

Immediately after, using the **Inset Faces** command, the new face was scaled inwards again but independently from the rest of the mesh body, defining the rim of the pot and the space available for positioning the soil. The latter was obtained by generating a circle filled via **Face Fill**; its position was set within the opening created through extrusion.

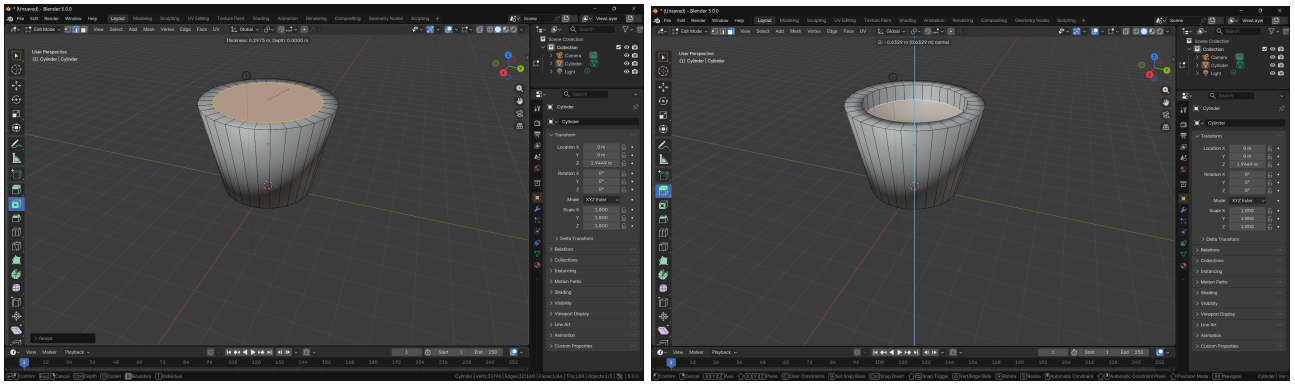


Figure 21: Opening the pot

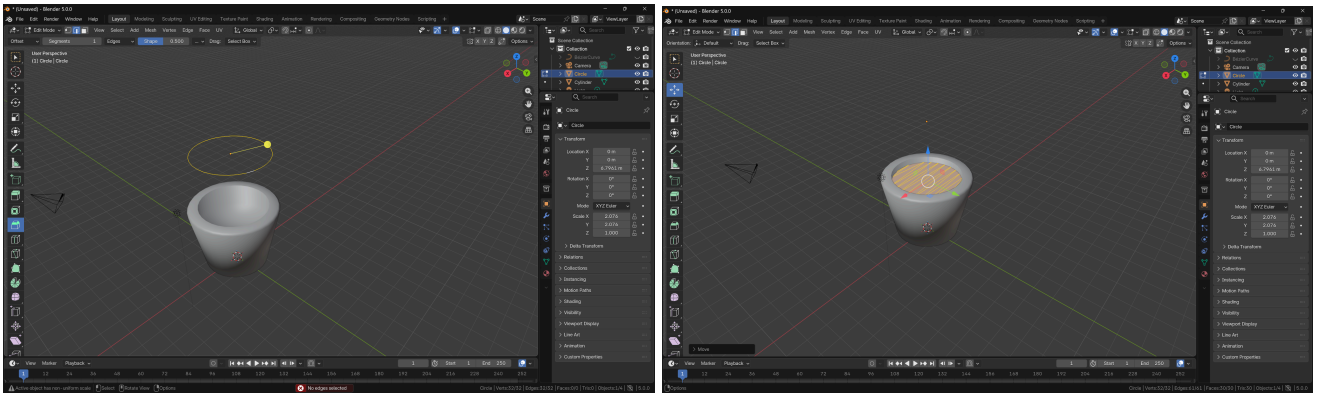


Figure 22: Circle fill face to create the soil

To conclude the modeling of the assistant's only non-organic element, the **Bevel** command was applied to the external and internal vertices. Through the increase of vertices, this allowed for the creation of **rounded corners**, giving a softer and friendlier perception to the appearance of our assistant's base.

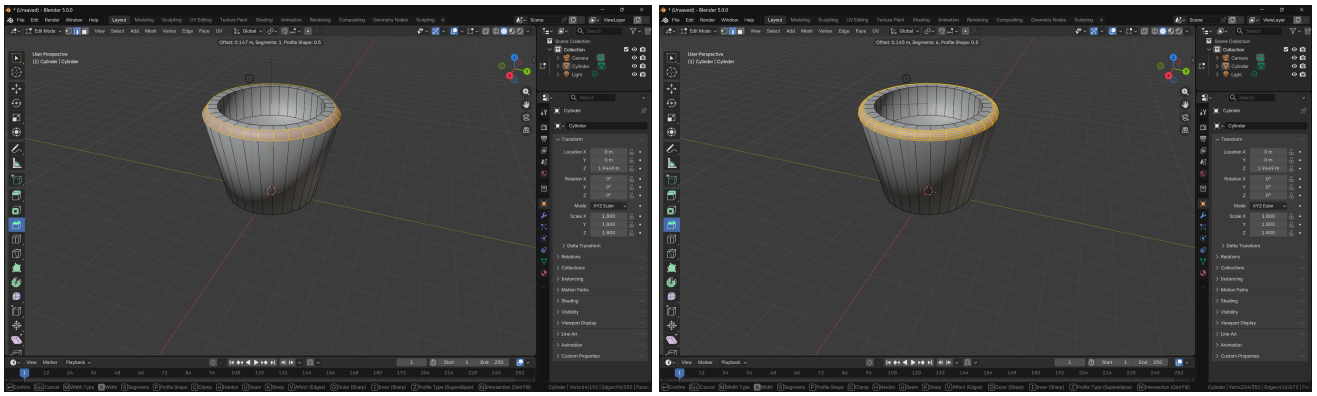


Figure 23: Edges creations with bevels

The rest of the model was obtained through the modeling of **organic elements**, developed according to a **Box Modeling** approach. In this specific case, the volume extrusion phase was often replaced by local mesh deformation along a single cartesian axis.

Regarding the stem, it was created by generating a Bezier curve. A depth level was set in the geometry properties to generate adequate thickness for the stem. However, the shape was first defined by moving and adjusting the control points (particularly in the upper part) to give the element an appropriate curvature. After the correct modeling of the curve, to assure the full compatibility with the animation pipeline of Unity, the Bezier curve was converted into a standard mesh.

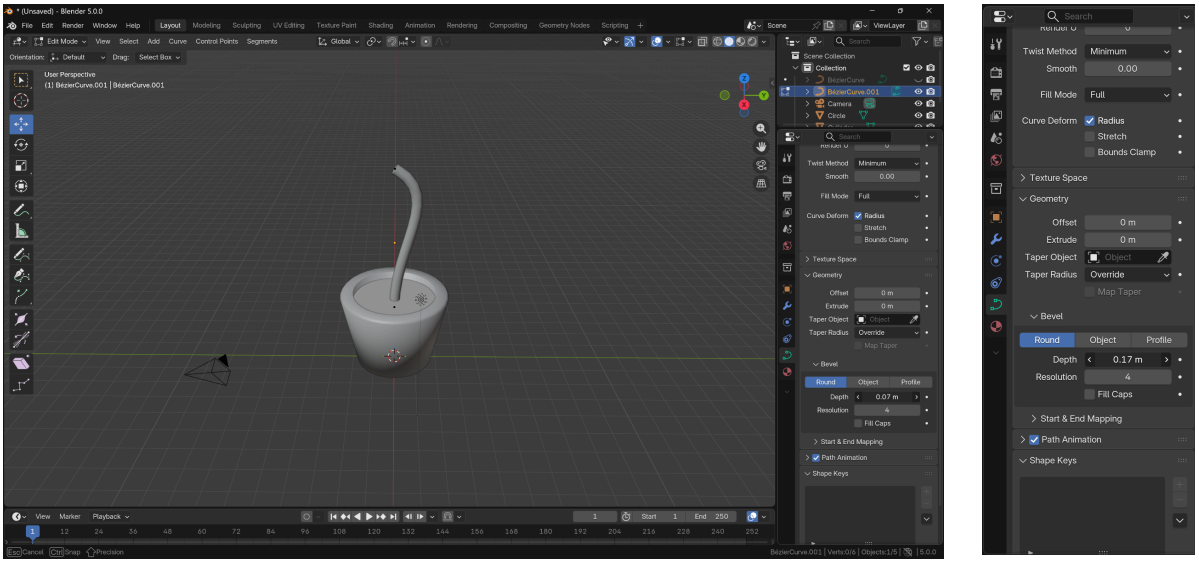


Figure 24: Creation of the stem

A similar **non-uniform deformation approach** was used for the leaves, corolla, petals, and eyes. For the first two, we started from a simple **UV Sphere**, which was then deformed with a **Scale** operation along the X-axis; this first transformation served to create the adequate thickness of the leaf, while the second was a longitudinal deformation along the Y-axis to give the element its necessary length. The head followed the same approach, but the non-uniform deformation was applied in only one direction.

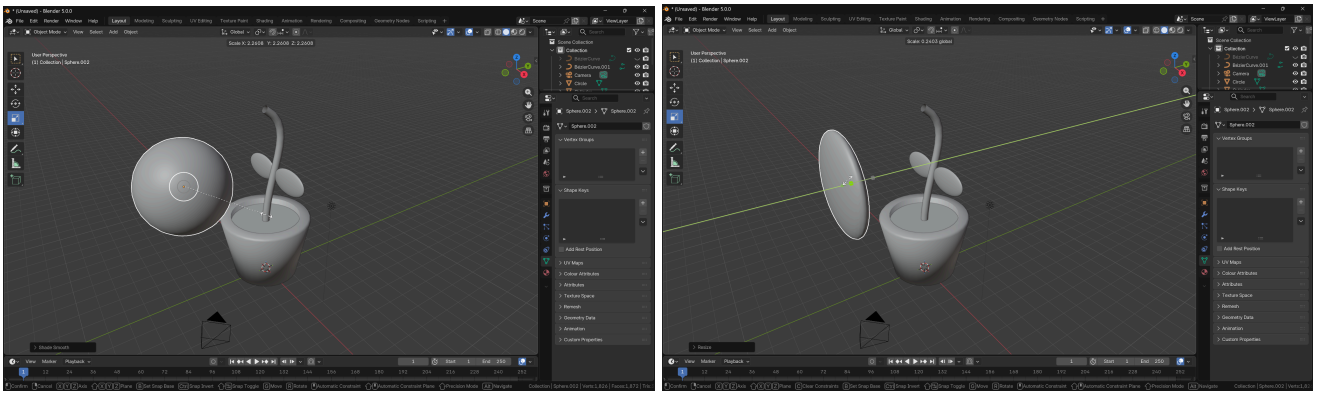


Figure 25: Modelling the corolla

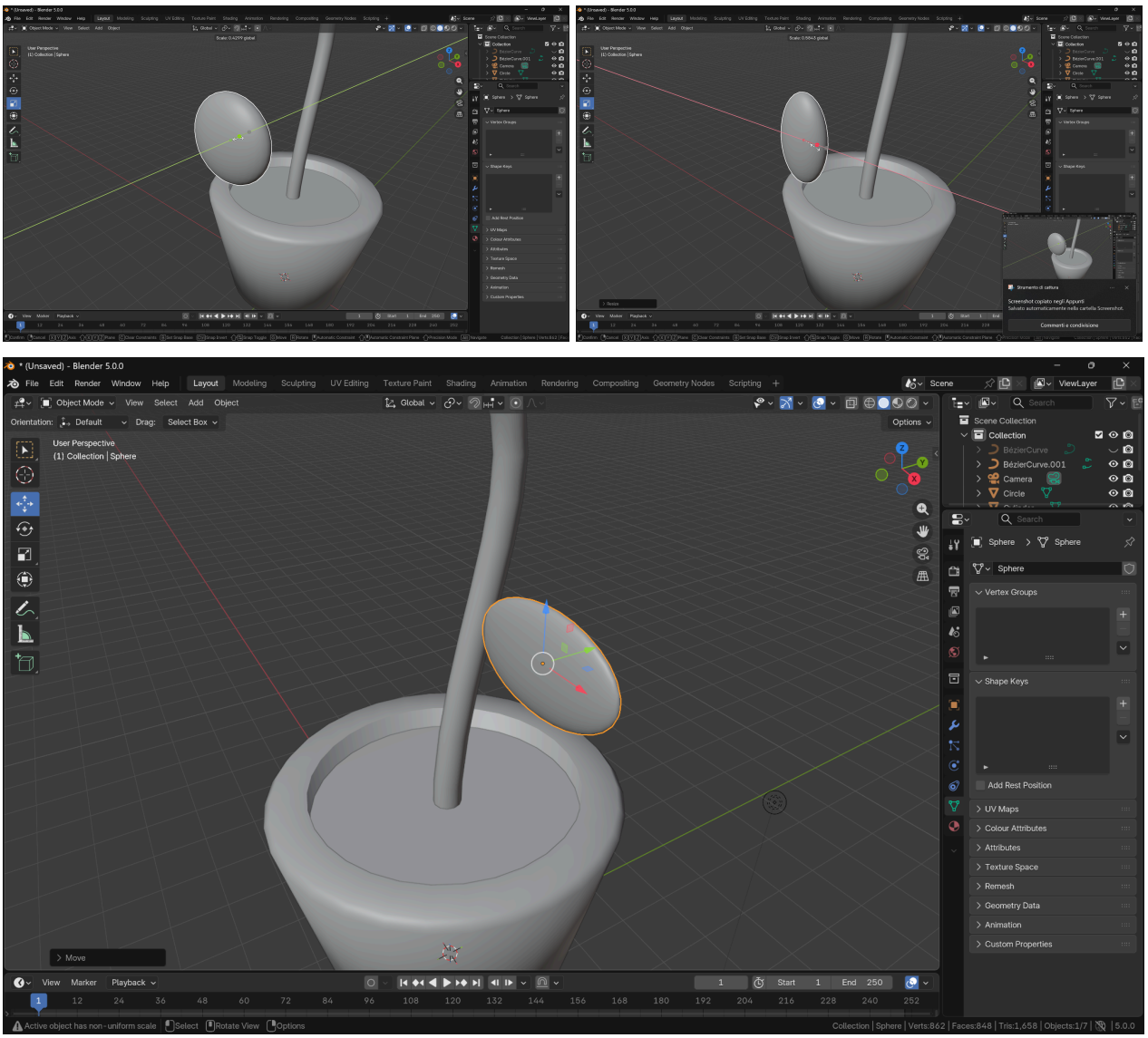


Figure 26: Modelling the leaves

The **petals** were modeled starting from a cylinder deformed non-uniformly along the Y-axis. A **Bevel** was then applied to the upper vertices, with a corresponding increase in geometry on the new inclined surface to achieve the desired curvature. The elements were then duplicated and arranged **radially** around the corolla.

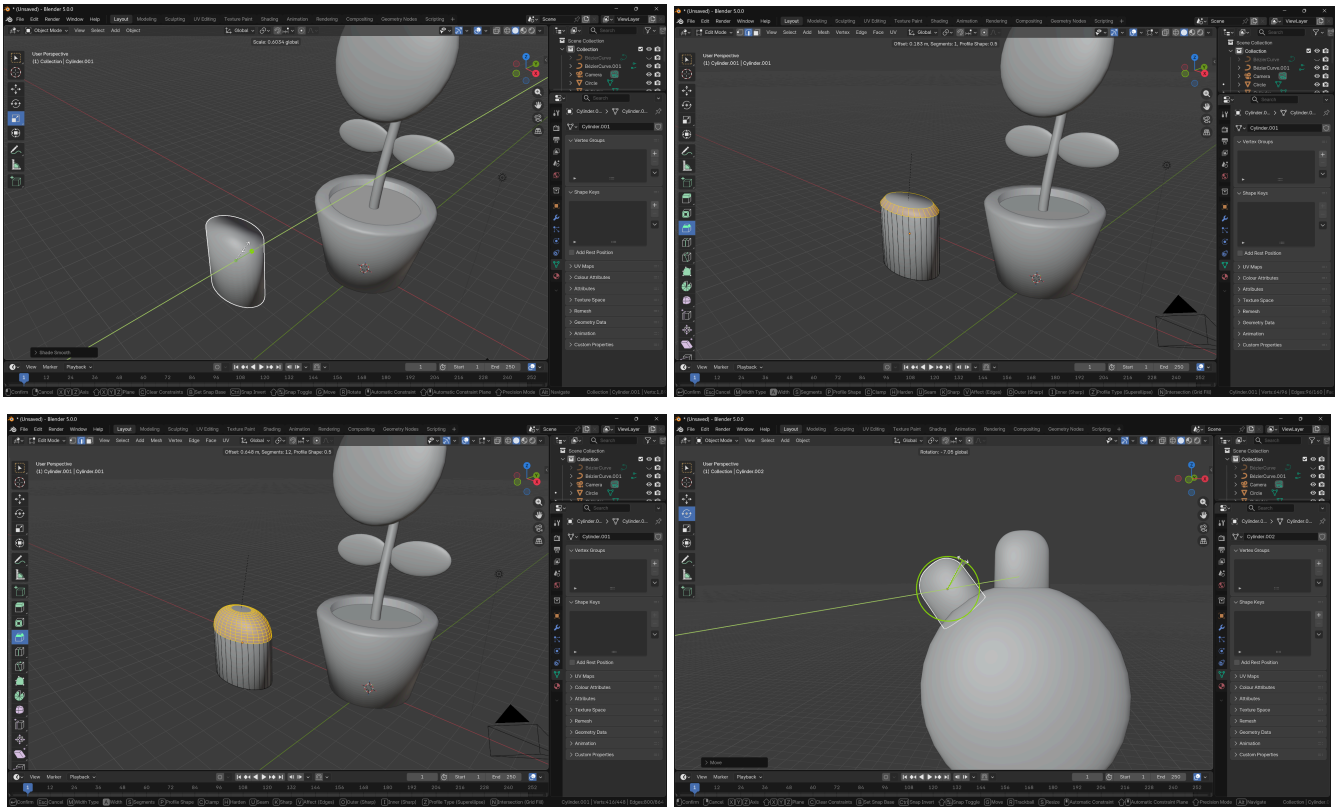
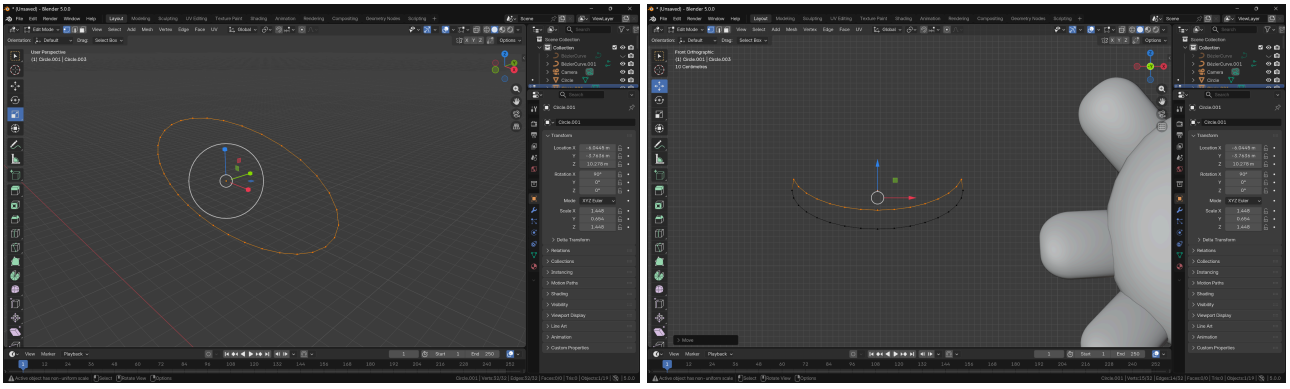


Figure 27: Modeling and positioning of the petals

For the **mouth**, the process was perhaps the most complex. It is a mesh obtained by generating and deforming the vertices of a circle. These vertices were rotated and aligned on the same plane and subsequently translated along the Cartesian axes to approximate the shape of a **semi-open smiling mouth**.



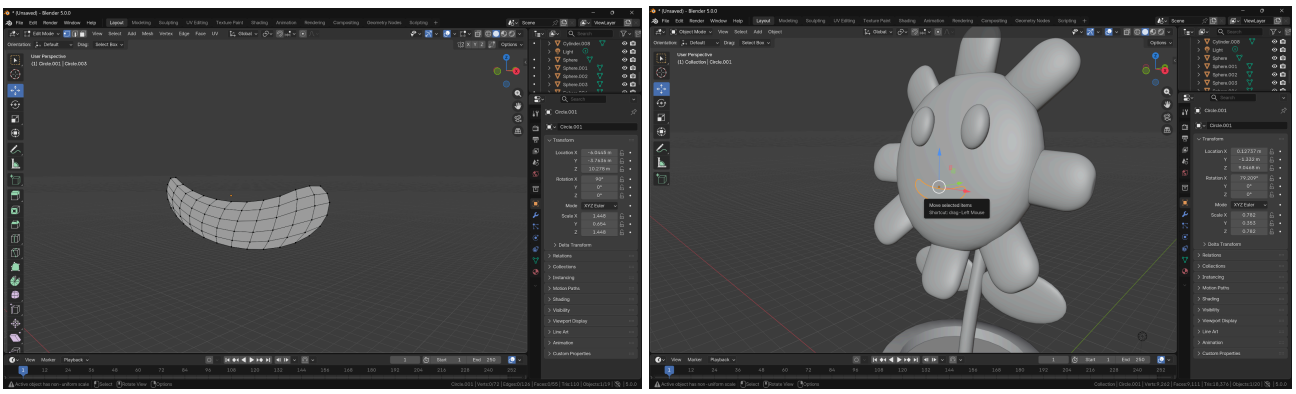


Figure 28: Modelling and positioning of the mouth

Once the initial mesh was complete, the next step was to bring the geometry to a form that matched the optimization constraints of a VR pipeline while preserving the assistant's silhouette and readability. This was achieved by using Blender's Decimate modifier, a tool that reduces geometric complexity by decreasing faces, vertices, edges, and triangles through a controllable reduction factor. The modifier was applied while paying attention to the mesh's density and topology, so that simplification did not compromise the features that make the character immediately recognizable. Working through modifiers also enabled a non-destructive workflow: changes could be iterated and evaluated progressively rather than "baked" prematurely, allowing the mesh to converge toward a Unity-compatible asset without losing control over its structure.

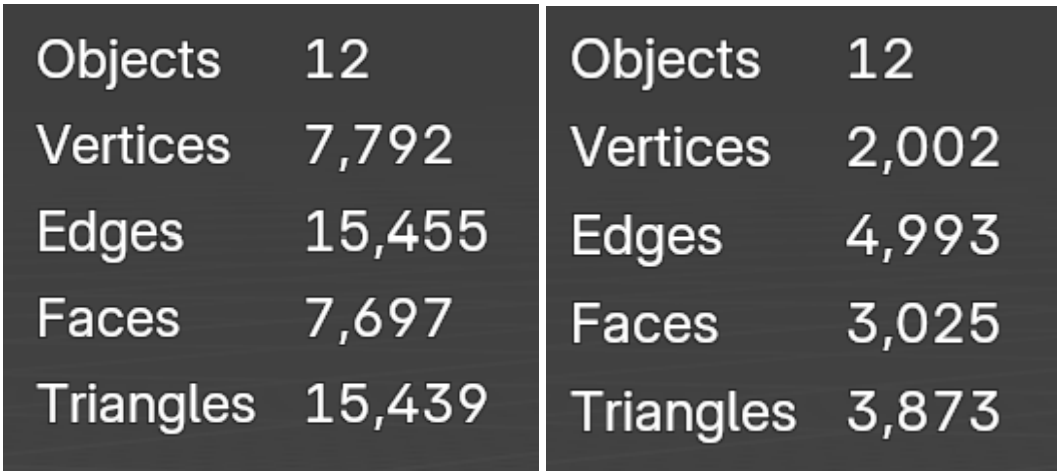


Figure 29: Model features before and after applying the modifiers

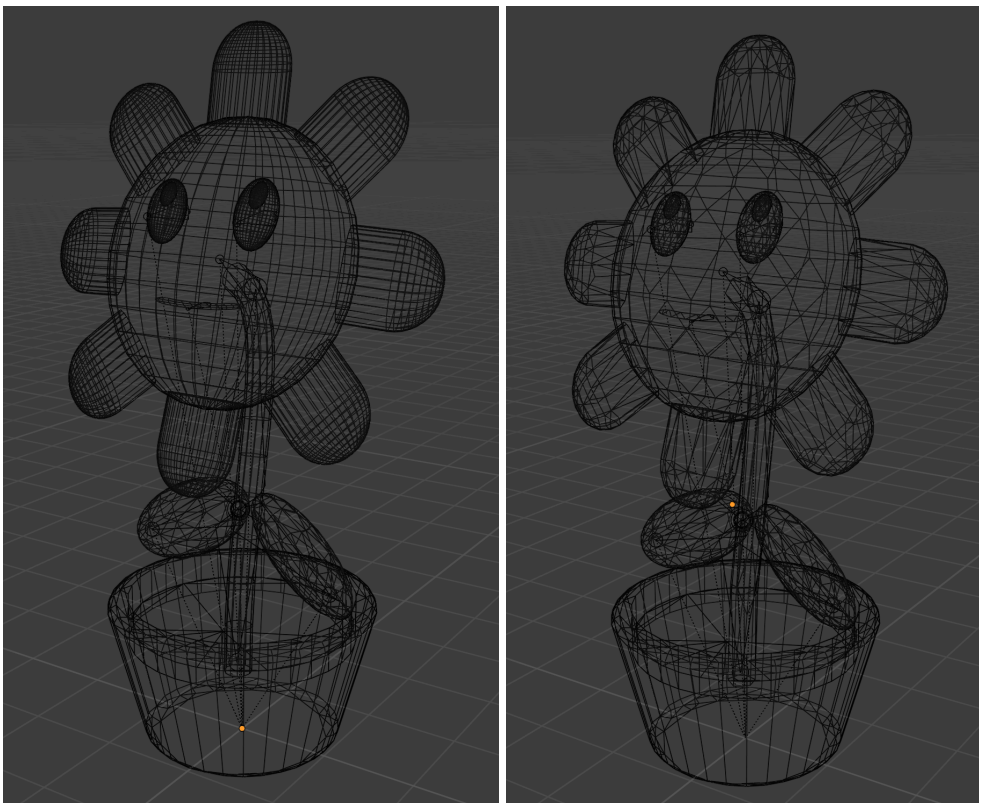


Figure 30: Simplified wireframe of the model

Animation rigging

The most crucial step when it comes down to animating a 3D model is building the **rig**, a control structure for animation that drives the 3D model: it is essentially a **simplified skeletal structure** that is composed of **bones**, where each bone is a node upon which 3-dimensional transformations can be applied such as rotations, translations, and scaling, in a controlled and repeatable way compared to frame by frame animation.

The rigging pipeline was built to be robust under real-time constraints and to integrate cleanly with Unity's animation system. To maintain low runtime latency and to reduce export complexity, the rig was intentionally compact, avoiding deformation-heavy solutions and limiting the number of bones to what is strictly necessary for readable motion.

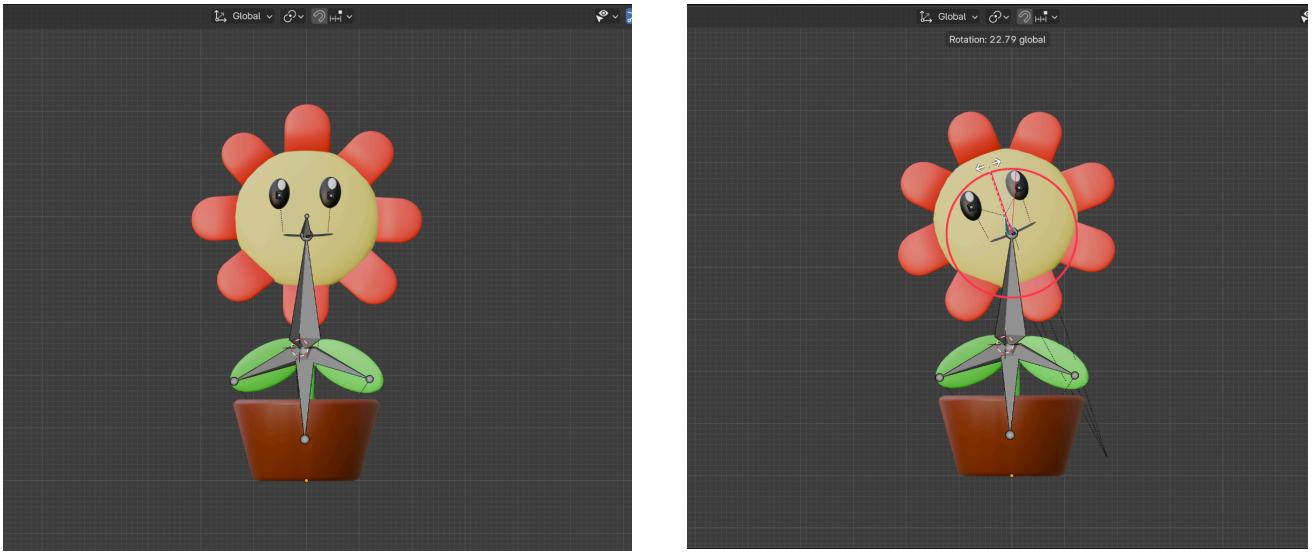


Figure 31: Seats' rig in Blender

A compact skeletal hierarchy was made in Blender, centered on a root bone that defines the assistant's global position and orientation, leaf bones that control the "arm" movements and a head bone that drives the main upper-body rotation. From the head, a small number of child bones control facial components such as the eyes and mouth. In this hierarchy elements automatically follow head motion while remaining independently animatable when needed.

To improve stability and eliminate common skinning artifacts, rigid facial features were treated as separate meshes parented directly to their corresponding bones rather than being driven through weight painting or shape keys, which can introduce inconsistent deformation or require more complex authoring.

Expressive motion is produced entirely through bone transformations, keeping the animation system uniform and lightweight. Eye blinks are implemented by scaling the eye bones, and mouth motion is driven by a dedicated bone to support simple open/close movement suitable for speech animation. Where necessary, constraints clamp rotations and scaling within safe ranges to prevent extreme poses that could create visual glitches.

Animation management

The animation processes were performed entirely in Blender using spatial transformations (rotation, displacement, and scale) recorded using Keyframes. In this sense, the use and implementation of the rig greatly simplified model management by avoiding the need to generate transformations and displacements that would have had to be managed separately in Unity,

potentially creating import errors or significantly increasing the complexity of inserting and connecting the model to the prototype.

By setting the Blender interface to Pose Mode, the rig could be manipulated correctly, even allowing the transformation pivots of the individual meshes composing the object to be changed using Blender's interface functionality. After moving the individual parts using the rig, thanks to the insertion of Keyframes, the program notes the change in coordinates of the mesh and allows the user to move from the previous position to the next within the window of time between two keyframes in the timeline. Based on this logic, all the meshes were animated to generate the interactions described in section 4.1.3. In the cases of Idle and Cheering, keyframes were arranged in the timeline in repetitive patterns that allow the action to be adequately extended for as long as necessary.

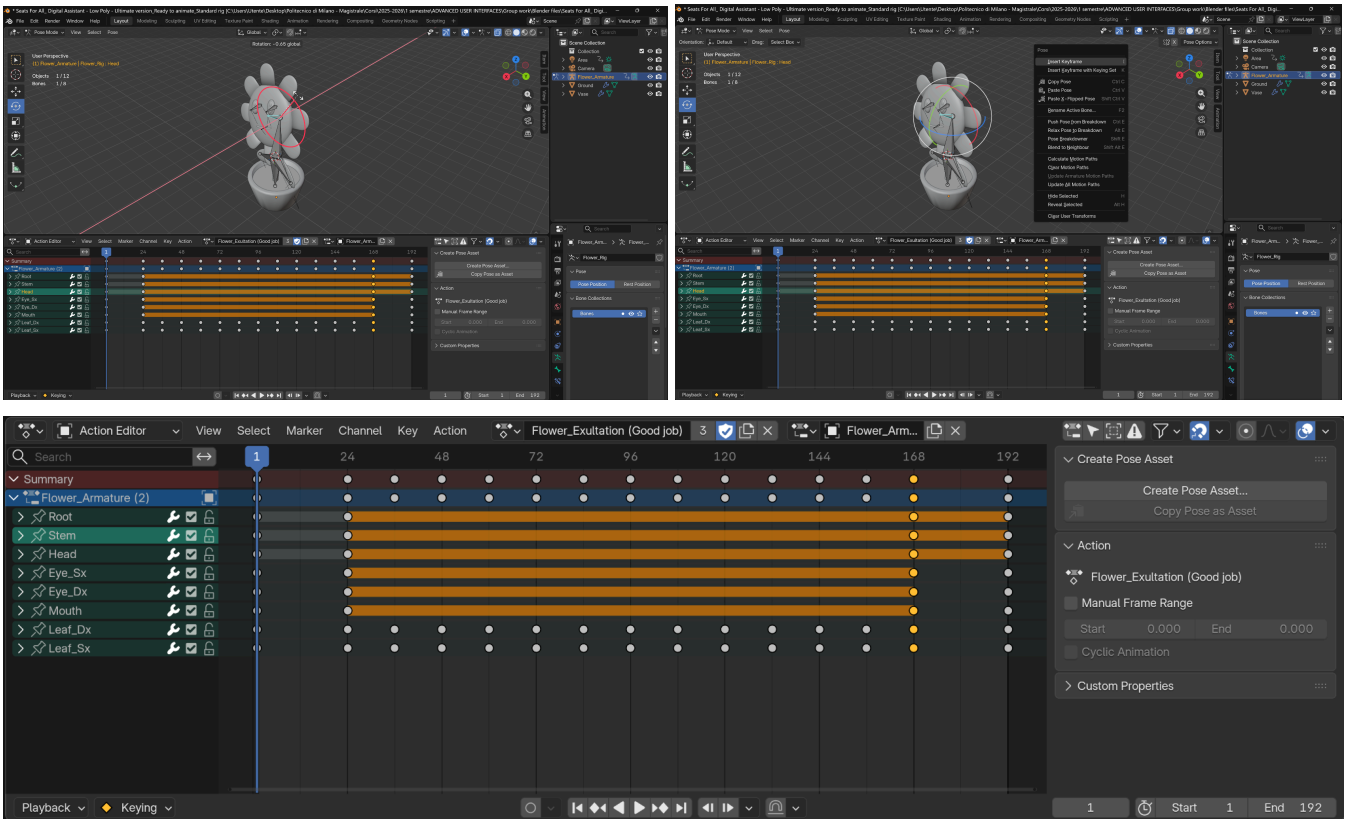


Figure 32: Transformations and keyframes

Each animation was created as a separate action, allowing for the independent development and management of the distinct behaviors This structure supports a clean, modular animation pipeline, where individual actions can be refined, tested, or replaced without affecting the others.

Actions were carefully named and organized to facilitate correct export and identification of each component when exporting to Unity.

5.2.2 Artificial Intelligence - Microsoft Azure

The design constraints of the application made it clear that a single model would be insufficient: Seats must sustain continuous spoken interaction while also performing visually grounded reasoning on Dixit cards. These two requirements impose different latency, context, and reliability constraints, and they operate on different input modalities. For this reason, Seats adopts a dual-model architecture deployed on *Microsoft Azure*, where each model is assigned a specialized responsibility and the overall system is engineered to keep the interaction **robust** under real-world VR conditions in a context where multiple participants are involved and can cause disturbances, but still maintain **low response latency** to feel as if they were another person at the table.

The AI system is therefore decomposed into two cooperating components. The first is a real-time conversational model responsible for dialogue, phase handling, and the orchestration of game actions, Seats' "voice". The second, is a completion model responsible for interpreting captured visual frames, identifying Dixit cards, and producing grounded descriptions that the first model can reason on, Seats' "eyes". The following subsections outline the functional responsibilities of each component and describe the communication strategy used to coordinate them.

Realtime - Seats' voice

The Realtime component represents the conversational core of Seats, responsible for continuous spoken interaction, turn management, and the orchestrating of game actions. Rather than treating the assistant as your usual "chatbot", the *Unity* implementation frames Realtime as an event-driven service that receives streaming audio, produces streaming speech output, and emits structured function calls whenever an in-game action must be executed. This design is motivated by the constraints of VR usage, where latency, interruptions, and partial utterances are frequent, and where the system must remain responsive without compromising phase correctness.

Due to the need for a larger context window to handle an approximately 30 minute play session, thoroughly worded procedures in the prompting that would be remembered by the assistant, and the availability of seamless vocal communication, the `gpt-realtime` model was chosen for deployment, which supports realtime events to communicate between the client and server, as detailed in the Azure documentation [11].

1. **Initial Connection**

At the transport level, the connection to *Azure OpenAI Realtime* is handled by the `RealtimeSocketClient` class. The client establishes a persistent WebSocket session using the `realtime` subprotocol, targeting the Azure Realtime endpoint with the selected deployment and API version, which lasts the entire playing session.

```
public async Task Connect()
{
   _ws = new ClientWebSocket();
   _ws.Options.AddSubProtocol("realtime");

   string uri = "endpoint + key”

   _cts = new CancellationTokenSource();

   try
   {
       await _ws.ConnectAsync(new Uri(uri), _cts.Token);
       _ = ReceiveLoop();
       OnConnected?.Invoke();
   //. . .
}
```

Listing 4: Connection to Realtime OpenAI through WebSocket

Once the connection is established, the behavior of the assistant is determined through a session-level prompt and tool configuration injected by the `MessageManager` class, which also handles all the event responses being sent by realtime

The initial prompt is extensively thorough by design. Our goal was to have an assistant that was aware not only of the game state, but also of how exactly every game state would unfold, what the child could ask for assistance with, which tools are accessible in every phase, on top of the contextual information of the application such as its target user and the environment. The vastness of the prompt mainly comes from the `procedures` string, which contains a detailed explanation of how the system should behave in every phase and what it should expect.

```
"PROCEDURE: Initial Hand Complete
When you receive [INITIAL HAND COMPLETE]:
1. Congratulate the player on scanning all 6
cards
2. You will receive the complete hand with
all card descriptions
3. Explain what to do next based on their
role:
```

```
"IMPORTANT - INTERPRETING 'THIS CARD':
When child says 'this card', 'that card',
'what about this', 'how does this connect':
- They are PHYSICALLY POINTING at a card
- IMMEDIATELY call activate_detection to
scan it
- Do NOT try to guess which hand card they
mean
```

```
  - If Narrator: Choose one card from your hand and think of a word or phrase that relates to it
  - If Guesser: Wait for the narrator to announce their word
4. Be concise and clear
5. Do NOT offer card suggestions yet - wait for them to ask"
```

```
- Examples:
* 'What about this card?' → Call activate_detection
* 'How does this connect to the word?' → Call activate_detection
* 'I'm not sure about this one' → Call activate_detection
* 'Does this match the narrator word?' → Call activate_detection"
```

Listing 6: Examples of procedures

The procedures are concatenated with an introduction to the game, a summary of the rules, the playing environment, the response style of the agent, and the available function calls. And the first payload is sent through the WebSocket to set up the session.

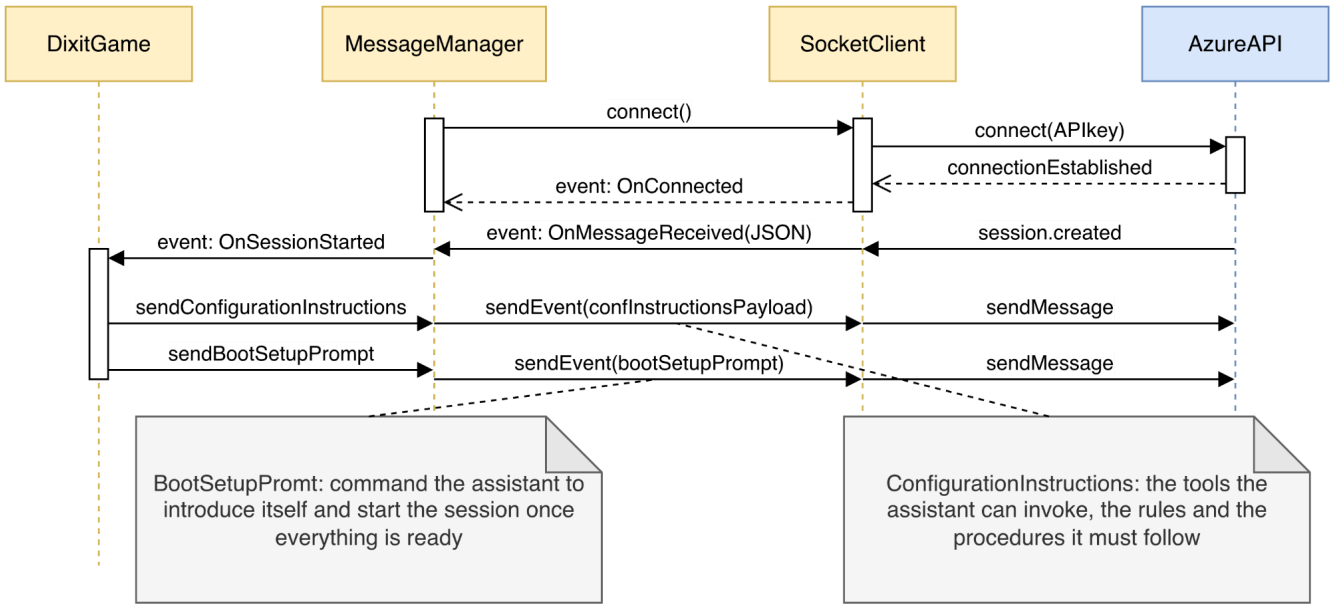


Figure 33: UML sequence diagram of game session setup

2. Message Handling

At runtime, all outgoing and incoming event messages are handled within the same class. Realtime emits an asynchronous stream of JSON events, any of which may arrive fragmented or out of order relative to Unity's frame loop. `MessageManager` is designed around this constraint: it treats the socket stream as an event bus and the `handleMessage` method turns the raw

event-based stream into deterministic game-side behavior through a sequence of if-statements that react to the events' `type` accordingly. The method's structure highlights the design choices of system:

- **`input_audio_buffer.speech_started:`** designed with **user speech priority** at its forefront, this event stops the playback of audio from the realtime assistant the moment the user starts talking, with the intent of creating an experience that never overwhelmed the user and where they feel listened to. In addition, this event also updates the last time the user spoke, which is kept track of in the system so the agent is able to give the user friendly reminder of their presence if they haven't spoken for 1 minute.
- **`response.audio.delta/done:`** the first event prioritizes **immediacy** of responses, parsing audio chunks as soon as they are synthesized. The audio is played incrementally by storing the deltas into a buffer and beginning playback once a full json has been received as a **robustness** measure. The second event further strengthens the measure by marking when a full response has been parsed, and enabling multiple responses to be queued in the buffer without the assistant ever cutting off.
- **`response.done:`** marks the end of the steam, the response's JSON is checked for any function tools that the assistant is calling. It treats tool execution as something that should occur only once the model has finished producing its response, which avoids interleaving tool side effects with the ongoing streaming output.
- **`conversation.item.input_audio_transcription.completed:`** one of the most important constraints in always-on voice systems is false activation, especially in environments where the user is expected to communicate with others. This method addresses that with this strict gating step. Instead of always generating a model reply when speech is received, it first checks the transcription of the user's speech. Only if the content of the utterance matches one of the identified *wakePatterns* does it send `type : response.create` allowing the assistant to respond.
- **`response.audio_transcript.done:`** once the agent produces a response and the final transcript is received, an *interaction window* is opened. This mimics the behavior of virtual assistants [12], that once awoken allow continued interaction with the user until manually deactivated. Because of the handsfree constraint of the systems, this interaction window is handled through time and syntactic based heuristics as mentioned in section 4.1.3.

3. **Tool Dispatching: Seats' hands**

Seats is able to talk and give instructions to the user, yet this is not enough, in fact, Seats also need to modify the state of the system. For example, if the child tells Seats the narrator word the system needs somehow to remember this word. Another example: if the child wants to update

their role (Narrator or Guesser) or register how many points they have gained, and they want the system to record such changes, they need a way to do so. This problem is linked to the fact that the child has no controller or button they can press to change the system (and this is something necessary since the child's hands are occupied in holding the cards), hence they have to rely on Seats to execute what they want to do.

Azure supports a certain structure of message called function tool. This kind of messages belong to the response.done category and they feature a set of customizable input values as well as a description of what the tool does. A function tool is therefore specified by:

- `name`
- `input fields`
- `description`

These three fields have to be formally specified in JSON format in what in this document will be called a *tool schema*. Each function tool has its own tool schema. All the tool schemas have to be sent in a unique `session.update` message (it has to be unique since the second session.update message will override the first). This will make the assistant aware of the tools it can invoke. Even though tools support a description, this is not enough to ensure that the tool will be invoked at the right moment, this is why additional prompting is needed for each tool (in the code *toolInstructions* strings were added to further specify when a tool should be invoked). Once Seats is ready to call a tool, it will send a response.done message (as described above) containing the name of the tool and the inputs. This message will also contain a unique id to identify that specific call.

On the client side, the system has to react to the function tool invoked to update its internal status. In our application, tool invocation messages are sorted by MessageManager, which also extracts the JSON body containing the name and the input fields. The JSON is then passed to another class called `ToolDispatcher` that performs the update to the system. To handle the complexity of a growing number of function tools we decided to implement the *Strategy programming pattern* [12]. In fact, each function tool is associated with a class named "Strategy", for instance `SetNarratorWordStrategy`. Each Strategy class has an `Execute` method that updates the system. During configuration, ToolDispatcher creates one instance of each strategy and keeps a dictionary indexed by the name of the tool that contains these instances. When `ToolDispatcher` receives a tool invocation by `MessageManager`, this dictionary is scanned and the `Execute` method of the strategy corresponding to the tool invoked is invoked [Listing 8].

```
public (string toolsSchemasString, string toolsInstructionsString) Setup()
```

```
{
    SetupTool(new SetNarratorWordStrategy(stateManager));
    SetupTool(new SetCurrentRoleStrategy(stateManager, phaseController));
    SetupTool(new SetNumberOfPlayersStrategy(stateManager));
    SetupTool(new AddCardDescriptionStrategy(stateManager));
    SetupTool(new FillInitialGameState(stateManager));
    SetupTool(new ActivateDetectionStrategy(detectionBridge));
    SetupTool(new DeactivateDetectionStrategy(detectionBridge));
    SetupTool(new SetPlayerScoreStrategy(stateManager, phaseController));
    SetupTool(new RemovePlayerCardStrategy(stateManager, phaseController));
    SetupTool(new StartScoringStrategy(phaseController));
    SetupTool(new ApplySetupStrategy(phaseController));
    SetupTool(new EndGameStrategy(phaseController));
    SetupTool(new RestartGameStrategy(phaseController, _reconnectSessionCallback));
    SetupTool(new QuitGameStrategy(_quitCallback));

    string toolsSetupString = $"\"tools\": [{string.Join(",", _toolSchemas)}]";
    string toolsInstructionsString = string.Join("\n", _toolInstructions);
    return (toolsSetupString, toolsInstructionsString);
}
```

Listing 7: Function tools setup

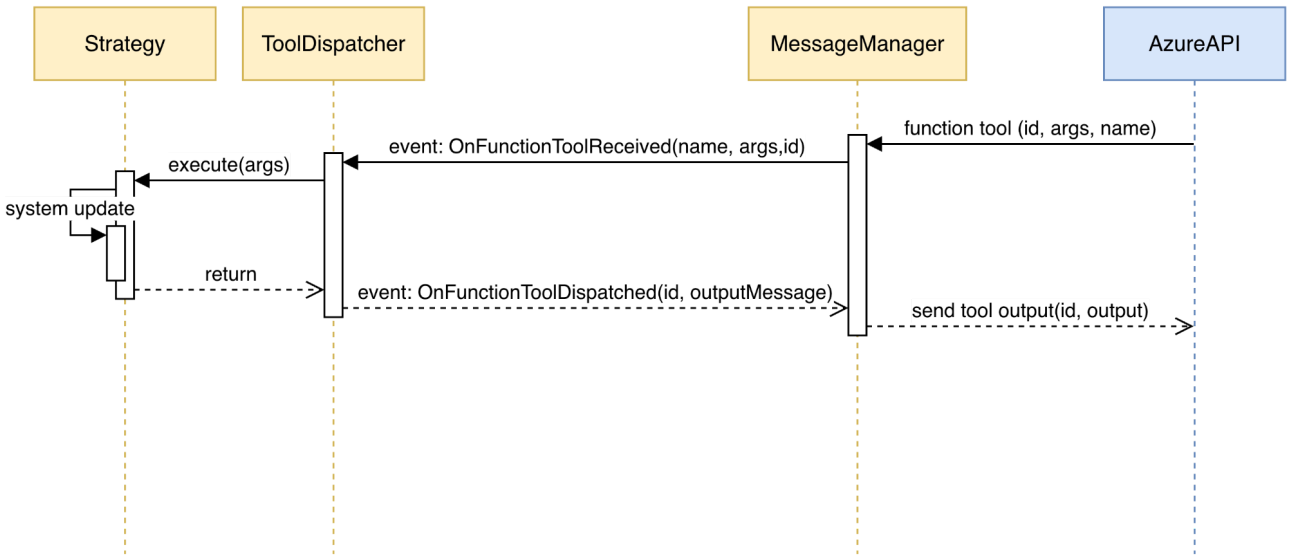


Figure 34: Function Tool Invocation UML sequence diagram

Finally, after having called a tool, the Azure assistant expects a response of type `conversation.item.create` in which another payload of type `function_call_output` is

nested. This payload has to contain the unique id of the function tool invocation and an output field containing the status of the invocation (for instance, success). The assistant will be completely unresponsive to vocal messages until this message is received.

GPT-4o - Seats' eyes

If Realtime provides the conversational interface of Seats, GPT-4o provides the perceptual grounding required by tabletop games. The assistant cannot meaningfully support children during the experience unless it can interpret what is physically present on the table and translate that into language. In a VR setting this introduces a distinct set of constraints compared to speech: camera frames are noisy, detections can jitter due to perspective distortion, and the system must avoid repeatedly capturing and uploading images in a way that would increase latency, cost, or user frustration. The GPT-4o integration was therefore engineered as a perception pipeline that favors stability, bounded payload size, and phase-aware behavior, rather than analyzing everything at all times.

Our decision was to deploy the `gpt-4o` [13] model provided by *Azure OpenAI*, which measures on traditional benchmarks, achieves fast and versatile performances for the tasks required by the system: Accepting text and image inputs and producing structured JSON outputs.

The model performs two main functions. Analyzing Dixit cards while checking them against the player's hand for duplicates, and extracting keywords in realtime's speech to give visual feedback to the user.

1. **Connection**

In contrast to Realtime, which requires a persistent WebSocket session to support bidirectional audio streaming and event-driven turn management, GPT-4o is integrated through a **stateless HTTPS request pattern**. This choice matches the role of vision in Seats: understanding images is only required at specific moments, after a stable card capture is obtained, and therefore benefits from isolated calls with explicit inputs and outputs rather than a continuously open channel.

The client performs an authenticated REST call to the Azure OpenAI endpoint for the GPT-4o deployment using the project's API key. Each invocation builds a compact payload containing either the schema-constrained prompt and the captured card image encoded in the multimodal message format supported by the API, or realtime's transcription, according to what is required by the system.

```
private UnityEngine.Networking.UnityWebRequest CreateWebRequest(string jsonPayload)
{
```

```
    var request = new UnityEngine.Networking.UnityWebRequest(API_ENDPOINT, "POST");
    byte[] bodyRaw = Encoding.UTF8.GetBytes(jsonPayload);

    request.uploadHandler = new UnityEngine.Networking.UploadHandlerRaw(bodyRaw);
    request.downloadHandler = new UnityEngine.Networking.DownloadHandlerBuffer();

    request.SetRequestHeader("Content-Type", "application/json");
    request.SetRequestHeader("api-key", openAIApiKey);

    return request;
}
```

Listing 8: HTTP POST Request

The model receives different prompts for its two purposes, one dedicated to image description and matching, and the other is speech summarization and extraction. In either case the most important prompting decision is the use of a *JSON-only contract*. The GPT-4o prompt does not ask for an open-ended description; it asks for a structured object response which the system can then use to update the game's context with a description or as visual feedback for the user.

2. Prompting Strategy - Card detection

The first prompt is sent whenever a card is scanned, and it defines the card description behavior of the model; should the ML detector have failed, it checks if the received image is a Dixit card, if not the scan is considered invalid.

If the card is recognized as a Dixit card, then the model attempts to match it with all the other cards' descriptions in the player's hand. If it is successful in doing so the structured object reports that its not seeing a new card and the same description it matched it with, otherwise the model is free to generate a creative description of the card.

The choice to send textual descriptions representing the player's hand to the model rather than using the images themselves was a compromise between the system's design goal to have **variety across every playthrough**, and the cost constraints imposed by the much more **expensive image tokens.**

```
private string systemPrompt =
    "Analyze image: Is it a Dixit card? Match against hand cards (IDs provided). " +
    "Return JSON: {\"cardId\":\"ID\",\"isNewCard\":bool,\"description\":\"text\"}. " +
    "NOT a Dixit card: cardId=\"INVALID\", isNewCard=false, description=\"\". " +
    "Matches hand card: isNewCard=false, cardId=matched ID, description=copy from hand. " +
    "NEW Dixit card: isNewCard=true, cardId=\"NEW\", description=2 sentences on
imagery/mood/symbolism. " +
```

```
    "JSON only, no other text.";
```

Listing 9: GPT-4o card description

3. Prompting - Summarization

The second prompts define an equally important aspect of the GPT-4o integration: **turning the assistant's spoken help into usable, immediate visual feedback** for the child. In practice, when a table card is scanned during a help request, the system expects the Realtime model to respond verbally with guidance; however, spoken explanations can be hard to retain in-the-moment, especially for our young target users possibly with attention disorders. For this reason, GPT-4o is also used as a *speech-to-structure* module: rather than "summarizing" freely, it is prompted to **extract up to three short clue items** directly from what the assistant *already said*, and to return them in a strict JSON payload `items:["...","...","..."]`.

To avoid variations between what the realtime said and the summarization, and optimize costs, the implementation has a bounded generation `max_tokens: 60` and deterministic behavior `temperature: 0`.

```
if (phase == DixitGamePhase.CardChoosing)
{
    systemPromptSpeech =
        "From the transcript, extract up to 3 clue items the assistant suggested. " +
        "Each item may be 1-8 words (short phrase ok). " +
        "Output ONLY as JSON: {\"items\":[\"...\",\"...\",\"...\"]}. " +
        "Use the transcript language. Do not invent text not in the transcript.";
}
else
{
    systemPromptSpeech =
        "From the transcript, extract up to 3 key imagery items that sum up the transcript
best (short phrases ok, 1-8 words). " +
        "Output ONLY as JSON: {\"items\":[\"...\",\"...\",\"...\"]}. " +
        "Use the transcript language. Do not invent text not in the transcript.";
}


string payload = "{" +
                 "\"model\":\"gpt-4o\"," +
                 "\"messages\":[" +
                 "{\"role\":\"system\",\"content\":\"" + EscapeJson(systemPromptSpeech) +
"\"}," +
                 "{\"role\":\"user\",\"content\":\"" + EscapeJson(transcript) + "\"}" +
                 "]," +
                 "\"max_tokens\":60,\"temperature\":0" +
                 "}";
```

```
//. . .
```

Listing 10: GPT-4o keyword extraction

Models Coordination

The two models are coordinated through the application as a strict handoff pipeline, rather than being allowed to freely influence each other. Realtime remains the single conversational channel, and GPT-4o is only invoked when the system needs a structured artifact that can be safely fed back into the interaction. This reduces inconsistent behavior during long, social play sessions and keeps responsibility clear: one model speaks, the other produces machine-readable evidence.

The handoff is implemented through *tool calls* and *event gating*. When Realtime decides vision is needed, it does not hallucinate visual details from dialogue. Instead, it calls for the function tools implemented by the classes `ActivateDetectionStrategy` and `DeactivateDetectionStrategy`, which route the request into `CardDetectionBridge`. From there, the vision pipeline runs only when explicitly activated, and `GPT4oCardAnalyser` is invoked only after a stable capture is produced.

Coordination is kept safe by strict output contracts. GPT-4o is prompted to return JSON-only results for both card analysis and transcript extraction, so its outputs can be validated and injected back into the Realtime.

5.2.3 Machine Learning Model - Roboflow & Ultralytics

The application required two computer vision detection models for its functioning. The first accomplishes the task of detecting the game board after the application launch. This allows the placement of the 3D assistant model in the virtual space. The model was necessary to '**embody**' the assistant right next to the board allowing the user to clearly see it, taking advantage of the Mixed Reality paradigm to grant the child a physical presence to look to for assistance and reassurance. Afterwards, board detection is disabled.

The second one is necessary to implement another core feature. This model, in fact, allows detection for dixit cards in proximity to the user. The reason why this is crucial to the application is that by tracking the card in space it is possible to scan it and send it to an azure assistant and receive a description. The process that led to the integration of these models into the application (from data acquisition to in-app testing) is the same for both models. Both models required datasets acquired by the team: the sample pictures were taken via mobile device and annotated using *Roboflow* [15]. Datasets were then exported to *Ultralytics Hub* [16] to be trained with

Google Colab [17]. The chosen model was *YOLOv8n* [18]. Finally the weights of the models were downloaded in ONNX format [19] and integrated in unity. In this section, all the steps belonging to this process are described in depth.

Data Acquisition

Training a detection model requires data, in particular annotated images. Since the objects that the team needed to detect are very peculiar it was impossible to find any useful dataset on the web. Therefore the team had to create the datasets first by taking many pictures of the desired objects and manually annotating them. However, the fact that those objects were very peculiar was also an advantage. In fact, this meant that a complex model was not required, so a very large dataset was not required either. For both models pictures had been taken with the same device: *IPhone 12 mini*.

For what concerns the card detection problem the team first took around 100 pictures of dixit cards belonging to the following categories:

- "Easy" samples: card easily visible on screen, good light, only one card in the picture, card not partially covered, card left on the table. This category made up around 70% of the set.

Figure 35: Easy samples

- "Hard" samples: cards 10% covered, more cards in the same picture, cards in hands. This category made up around 25% of the set.

Figure 36: Hard samples

- Negative samples: objects that resembled cards but were not cards (books, laptops, paper sheets, portraits and relevant pieces of furniture). This category made up around 5% of the set.

Figure 37: Negative samples

After training the team noticed that the model was performing poorly: hallucinations due to rectangular objects were frequent and the model often failed to detect cards in “hard” situations like more than one card on screen or cards held in hands. Thus, more “hard” context pictures and false negatives were necessary. In the end, the team took around 250 pictures, thus adding 150 more. In the updated version of the dataset the “easy” samples made up only around 30% of the

set while “hard” samples and negative samples made up around 30% and 40% of the set respectively. After training again, the model improved drastically: now hallucinations completely disappeared and detection was more robust, especially in “harder” situations in which more cards were held in hands.

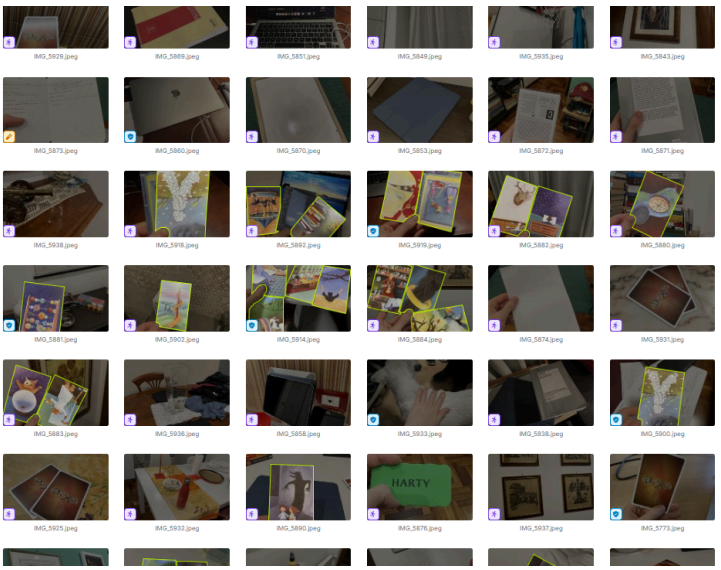


Figure 38: A small glimpse of the dataset

For what concerns the board detection problem, the team had already gathered experience from the first model; thus, this time only one version of the dataset was needed. The model should have detected the board even if cards were placed nearby and rabbits (the in-game marker) were placed on top of it. Another problem was light reflections caused by the translucent nature of the board. To address these issues the team took a total of around 270 pictures including:

- around 50 “easy” samples with the board in clear view on the table.
- around 40 more “easy” samples but with light reflections on the board.
- around 40 “hard” samples with cards next to the board and rabbits placed on top.
- around 40 more “hard” samples including reflections on the board.
- around 80 negative samples including books, laptops and empty table pictures.

Annotations and Augmentations

Roboflow was used to perform annotations for both datasets. Roboflow allows data transfer directly from mobile devices and has built-in support for annotations. Therefore, the team uploaded the pictures and manually annotated the samples. Annotations did not include just a bounding box but a polygon marking precisely the portion of the picture that was occupied by the

object. This allowed better performance and more precision, but it also took much longer to annotate.

Figure 39: Roboflow Logo

Figure 40: Hultralytics Hub Logo

As a final step, once all the annotations were ready, augmentations were added to both datasets. For both datasets, augmentations consisted in rotations, blurring, noise, hue and brightness alterations but they did not include flipping or mirroring. In total, augmentations almost tripled the number of samples, being now both around 600 in number.

Training and Integration in Unity

Roboflow allowed training but only with a very restricted range of models and with limited computational capabilities -that required a paid subscription once depleted. Moreover, Roboflow lacked customization, for instance it was impossible to choose the number of epochs. The first card detection model, the one trained on the "small" dataset described above, was in fact a *YOLOv11* model trained on roboflow. After noticing the poor performance of this model, acknowledging the limitations of roboflow and realizing that model weights could not be downloaded without a paid subscription the team concluded that training required another tool.

Then another solution was found, a website called Ultralytics Hub. This website could not train models without paying a subscription but allowed training through google colab. Generally, training via colab is not easy and requires loading the dataset before each session. However, Ultralytics Hub offered built-in support for google colab with automatic connection and code ready to execute. Therefore the whole training process was very easy and the usual frustration caused by colab was avoided. Moreover, Ultralytics Hub had a much wider range of models and allowed weights download, exactly what the team needed.

Therefore, for both models, datasets were downloaded from Roboflow and moved to Ultralytics Hub. For both models the choice was YOLOv8n, this fact is justified by the simplicity of the

model (around 3.2 million parameters). The advantages of having a simple model are the following:

- prevention of overfitting: since the datasets were in both cases small, a more complex model could have easily overfitted.
- performance: since the model is simple. predictions are faster and the application runs smoother.
- length of training: a simpler model is faster to train.

After training, both YOLOv8n models were exported in ONNX format and imported in Unity.

5.2.4 Game Engine - Unity

The whole application was built in *Unity* [26], one of the most popular game engines. The Unity version used for the project is version 6.2 (6000.2.7f2) with the addition of the several packages that were needed to create the XR environment complete with passthrough and tracking, run the card detector in the engine, and generally make easier the app's development
Unity is a powerful cross-platform game engine developed by Unity Technologies that offers a robust scripting environment utilizing C#. It is used to handle the game state logic, to make sure the assistants are always context aware in every phase of the game, successfully talking to each other and to connect the detection model to the VR environment.

What follows is a technical description of how each piece was implemented and the packages utilized to do so if applicable:

Figure 41: Unity logo

Game state logic
In Unity, the first thing we formalized was the game loop as a finite state machine, because SeatsForAll coordinates multiple subsystems that must never drift out of sync: the voice layer, the card detection pipeline, and the assistant-facing context that the LLM reasons over.

Rather than letting each component infer "what should happen next", we introduced an explicit phase variable `CurrentPhase` managed by a single controller `DixitPhaseController`,

and we constrained all progress to a known transition graph. The experience unravels over 8 distinct game phases, 3 starter phases that never repeat after initialization: `Boot`, `Setup`, `InitialHandScan`; and the remaining 5 which are necessary to keep track over the multiple rounds of the game `RoleCheck`, `CardChoosing`, `Voting`, `Scoring`, `DrawCard`.

Each transition between phases is validated centrally through the method `IsValidTransition`, preventing ambiguous flows and accidentally activating unintended features.

```
private bool IsValidTransition(DixitGamePhase from, DixitGamePhase to)
{
   switch (from)
   {
       case DixitGamePhase.Boot:
           return to == DixitGamePhase.Setup;

       case DixitGamePhase.Setup:
           return to == DixitGamePhase.InitialHandScan;

       case DixitGamePhase.InitialHandScan:
           return to == DixitGamePhase.CardChoosing;

       case DixitGamePhase.RoleCheck:
           return to == DixitGamePhase.CardChoosing;

       case DixitGamePhase.CardChoosing:
           return to == DixitGamePhase.Voting || to == DixitGamePhase.Scoring;

       case DixitGamePhase.Voting:
           return to == DixitGamePhase.Scoring;

       case DixitGamePhase.Scoring:
           return to == DixitGamePhase.DrawCard;

       case DixitGamePhase.DrawCard:
           return to == DixitGamePhase.CardChoosing || to == DixitGamePhase.RoleCheck;

       default:
           return false;
   }
}
```

Listing 11: How the GamePhaseController checks for valid transitions

Once the phases are modeled, a state object that could be observed by the subsystems was needed to maintain synchronization. The `DixitGameStateProxy` acts as the compact state representation we keep in memory: it stores the **local player's role**, the **current round stage**, the **number of players**, the **narrator's clue word** for this round, the set of IDs of **cards currently in hand**, and the **player score**. To make this reactive, the proxy exposes an `OnGameStateChange` event, so other scripts can subscribe to the changes they are interested in keeping track of. Finally, The `DixitGameState` class then wraps these fields with setters that always call `NotifyChange()`, ensuring that every change in the system is observable and that no hidden state updates occur silently in the background.

```
public enum DixitRole { Unknown, Narrator, Guesser }
public enum DixitRoundStage { Unknown, CardChoosing, NarratorCardGuessing,
PointsComputation}

[Serializable]
public class DixitGameStateProxy
{
    [SerializeField] protected DixitRole currentRole;
    [SerializeField] protected DixitRoundStage roundStage;
    [SerializeField] protected int numberOfPlayers;
    [SerializeField] protected string narratorWord;
    [SerializeField] protected List<string> playerCards;
    [SerializeField] protected int playerScore;

    public static event Action<DixitGameStateProxy> OnInstanceCreated;
    public event Action OnGameStateChange;

    public DixitRole CurrentRole => currentRole;
    public DixitRoundStage RoundStage => roundStage;
    public int NumberOfPlayers => numberOfPlayers;
    public string NarratorWord => narratorWord;
    public List<string> PlayerCards => playerCards;
    public int PlayerScore => playerScore;

    protected void NotifyChange() => OnGameStateChange?.Invoke();
}
```

Listing 12: How the proxy keeps track of the game state

With the model complete, the succession of the rounds is defined using the rotation of player roles and the score of the user. In `DixitPhaseController`, the system keeps track of who the narrator is through the trio of variables `numberOfPlayers`, `localPlayerIndex`, and `currentTurnIndex`, where the narrator is simply the player whose index matches the current

turn. This can only be computed from the moment the system knows that the user is a narrator, a check that is performed by the realtime LLM in its specific phase, until the current user actually becomes the narrator. This allows the role to be written into the shared state through `gameStateManager.SetCurrentRole(...)` exactly when the experience enters `CardChoosing`, and then updated again each round by advancing the index of the narrator `AdvanceTurn()`.

Scoring is handled in the same explicit manner: when the scoring window closes with the notification `NotifyScoringFinished()`, Unity performs the cleanup steps: resetting the narrator clue with `SetNarratorWord("")`, incrementing `completedRounds`, advancing the narrator turn only if `narratorTurnOrderSynced` is true, and finally transitioning into the `DrawCard` phase back to the beginning of the new round.

```
public void NotifyScoringFinished()
{
    if (gameStateManager != null && gameStateManager.CurrentGameState != null)
        gameStateManager.SetNarratorWord("");

    completedRounds++;

    if (narratorTurnOrderSynced)
        AdvanceTurn();

    TransitionTo(DixitGamePhase.DrawCard);
}

private void AdvanceTurn()
{
    if (numberOfPlayers <= 0)
    {
        return;
    }

    currentTurnIndex = (currentTurnIndex + 1) % numberOfPlayers;
    localCardPlayed = false;
}
```

Listing 14: Advancement of turns

Meta and XR

Since the hardware implementation is centered around a Meta Quest 3 device due to the features previously mentioned, we utilized the Meta ecosystem to the fullest by importing their Unity compatible package SDKs, all of which sped up and facilitated the implementation of the project

in every aspect. More precisely, all Meta SDKs were installed in version 81.0.0.0, and the specific ones needed for the application are:

- **Meta MR Utility Kit** [28]: Provides a set of tools and components to simplify the development of Mixed Reality experiences, such as scene understanding and surface positioning.
- **Meta XR Core SDK** [29]: Contains the essential base APIs and foundational features required to build any virtual reality application for Meta Quest devices.
- **Meta XR Interaction SDK** [30]: Offers a comprehensive library of high-quality interaction patterns, including hand tracking and controller-based gestures.
- **Meta XR Interaction SDK Essentials** [31]: Includes the core assets and access to Meta's **building blocks**. These are pre-programmed, functional blocks that can be drag-and-dropped into Unity to implement complex VR features with a click.
- **Meta XR Simulator** [32]: Allows developers to test and debug XR features directly within the Unity Editor without needing a physical headset.
- **Unity OpenXR Meta** [36]: implements support for Meta's OpenXR extensions, granting access to specific Meta Quest hardware components such as the depth preceptors and the ray casters.

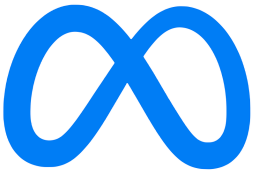


Figure 42: Meta logo

To enable XR development in Unity with AR/VR devices in the first place, the **OpenXR plugin** [37] was also installed to manage the application features that are required at runtime. This is the package that supports VR implementation in Unity.
All packages are all available directly from the **Unity Asset Store** [38].

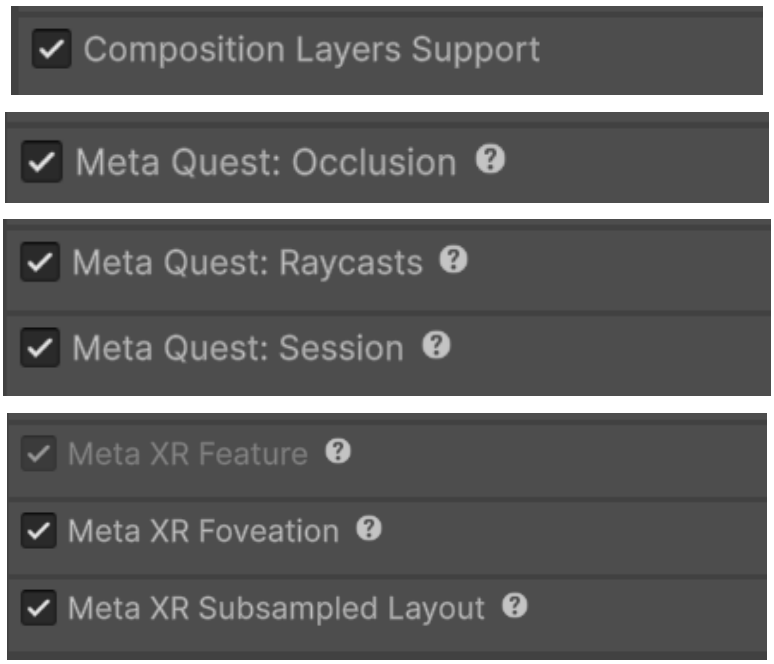


Figure 43: List of the enabled feature groups in the OpenXR Android build.

Inference Engine

Another very important package we used is **Inference Engine** [33], previously known as **Sentis**. Inference Engine allows machine learning models to be integrated into a Unity project in ONNX format.

The model trained on the Dixit cards is exported in the ONNX format and imported as a normal game asset in Unity, this model is then accessed by a class called `YOLODetector`. This class utilizes classes and methods from Inference Engine to build a predictor based on the weights in ONNX format. The real core of this app is a method called `TryToDetectCard`,this method takes an image in the Unity format texture and returns the bounding box of the object of interest spotted in the image.

The entire card detection system is handled by another class called `CardDetectionManager`, this class directly implements `YOLODetector`. In fact, `CardDetectionManager` has direct access to the headset center camera and to the ONNX model, which are both passed from the inspector along with the permissions to access the passthrough, the raycasts and the depth preceptors. The camera feed is passed to the method `TryToDetectCard`.

More specifically, the actions are performed in the following order:

- A 640 x 640 pixels portion of the main camera feed is taken. This is done to ensure that the dimensions of the image are compatible with YOLOv8 input standards. This step was done implementing a **shader**, so that the detection wouldn't overload the system by

creating a texture at every capture, which in testing has caused uncomfortable and disorienting lag spikes. The shader, located in the **Resources** folder of Unity's assets, is called `CropToSquareBlit`

- Then the image is sent to `YOLODetector`, a certain threshold of confidence is applied to prevent false positives.
- Then the 3d estimation phase, which is the most complex, occurs. During this phase the headset depth sensors are used to cast a ray from the camera to the center of the card and the four edges are used to reconstruct the 3d pose of the object in the space frmo the known dimensions of a Dixit card.
- In order to prevent the detection from "jittering", the class implements a kalman filter computed on the last 30 detections, a linear interpolation and an outlier detection is also applied to the prediction; all of this allows stabilization of the prediction in space rather than a constantly jumping marker at every slight movement.
- After this operation, the card is highlighted with a 3d outline marker, so that the user can see that the system is detecting the card.

```
private void ProcessCameraFrame(Texture cameraTexture)
{
    bool cardDetected = _detector.TryDetectCard(
        _yoloRT,
        boundingBoxTransform,
        Mathf.RoundToInt(confidenceThreshold * 100),
        debugText,
        out YOLODetector.DetectionResult bestDetection,
        out float detectionConfidence
    );

    Handle3DMarker(cardDetected, bestDetection);
    HandleCapture(cardDetected, bestDetection, detectionConfidence, cameraTexture.width,
cameraTexture.height);
}

private RenderTexture BuildYoloInputRT(Texture cameraTexture)
{
    float w = cameraTexture.width;
    float h = cameraTexture.height;

    float scaleX = 1f, scaleY = 1f, offsetX = 0f, offsetY = 0f;

    if (w > h) { scaleX = h / w; offsetX = (1f - scaleX) * 0.5f; }
    else if (h > w) { scaleY = w / h; offsetY = (1f - scaleY) * 0.5f; }
```

```
    _cropMat.SetVector(ScaleOffsetID, new Vector4(scaleX, scaleY, offsetX, offsetY));
    Graphics.Blit(cameraTexture, _yoloRT, _cropMat);
    return _yoloRT;
}
```

Listing 15: The core of the inference loop

Furthermore, this class commands another one called `CardCaptureManager` which is responsible for cropping the card from the image and capturing. The main processing steps performed by this class are the following:

- The camera feed alongside with the detected card are passed from CardDetectionManager. In order to stabilize the detection, the last 30 detection instances and their own bounding boxes are taken and an "Average bounding box" is computed.
- Then a 15% padding is added to the image to ensure portions of the card are not cut out.
- Finally, a new image of smaller size is created by copying the pixels from the original image provided by `CardDetectionManager`. This operation is performed in GPU memory.

Once these operations are completed, the new cropped image representing the card is optimized further by changing its format into a JPEG, compressing its size to 256x256 pixels, and reducing the quality of the image to 70% of the original. The choice was made purely for optimizations of costs, due to how expensive image processing tokens are in the Azure API.

Animation Integration and Playback in Unity

Once the three animations of our model were finalized in Blender, we exported the character as an FBX specifically to preserve the **rig-driven** animation in a form Unity can understand without ambiguity. The rig is the critical bridge here: because the motion was created as bone transformations rather than mesh transformations, Unity imports each Blender action as a **discrete animation clip** associated with the same skeletal hierarchy. This avoids the fragile scenario where animations depend on object-level transforms that can be misinterpreted during import, especially once the model is re-parented or re-scaled inside a Unity scene.

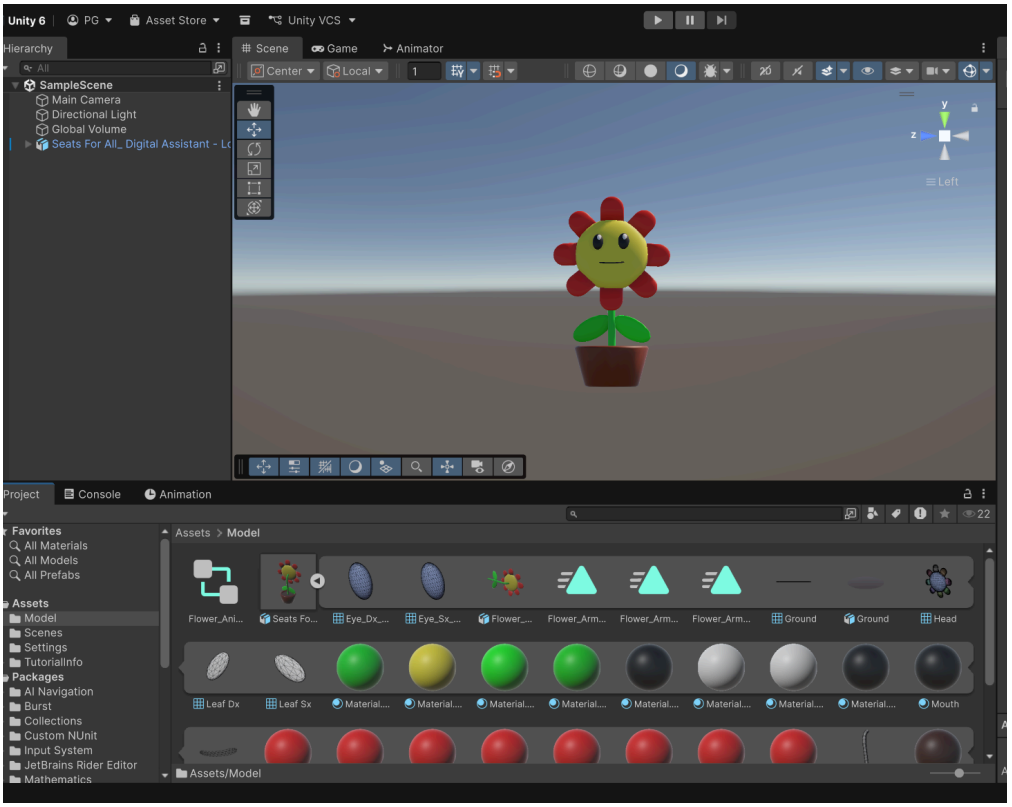
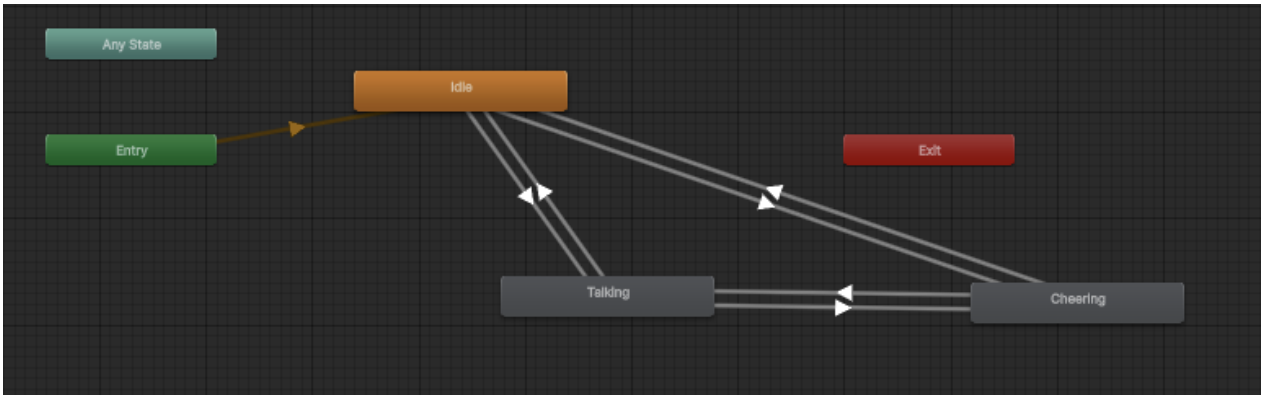


Figure 43: Import of the FBX file in Unity

On the Unity side, we configured the model as a **Generic rig** (appropriate for stylized non-humanoid characters like Seats) and bound the imported clips to a single **Animator Controller**. The controller handles the transitions between the animations using two booleans, `isTalking` and `isCheering`, and decides which animation should be playing depending on game context. To integrate the animations with the rest of the application, state changes happen through explicit runtime calls that change the aforementioned boolean values.



Figures 44: Setup of the animation controller

In practice, the assistant's behaviour layer sets animator parameters to switch between Idle, Talk, and Cheering according to game events: Idle is the default starting state, and also the default returning state when both booleans become false. Talk occurs whenever there are still elements inside of the speech buffer of the assistant, meaning the assistant is still talking, and Cheering occurs at the end of the scoring phase to celebrate with the child.
Cheering has greater priority over talking, and is the only non looping animation of the three. Once the animation is entirely consumed, the model will return to Idle of Talking depending on the value of the boolean.

Useful external tools
The **Quick Outline** [39] package was imported from the Unity Asset Store rather than implementing our own custom outline renderer. Its only script `Outline.cs` already achieved our primary objective of having an identifiable marker tracking our card in the 3D space.
Only two modifications were made to the script, one on the script itself to utilize a gradient of two colors instead of a single flat color onto the material, and the second was done to the `OutlineFill.shader` to achieve a scanning effect on the outline that matched the visual message of the system.

The versioning tool used by the team is **Github** [27], probably the most used. This was done to keep track of the history of the project, and keep the implementation of new features on separate development branches so that any mistake could be reversed seamlessly.

6 Value Proposition

6.1 Value for Users and Uniqueness

Seats for all is an ambitious project that aims at fully supporting a Dixit game session through an LLM assistant, the integration of this system into a MR visor is very unique. Another approach that would have satisfied the requirements of section 3 could have been the integration of the assistant into a "virtual" version of Dixit implemented into a mobile app or website. This approach would not have required detection models and the general problem of making the assistant aware of "what is going on": the game status. The experience with this kind of system would therefore be much smoother and the overall assistant behavior much more precise. However, we believe that this approach, although satisfying the goals in a better way, does not really fit the context in which the system should be used. In fact, table top games are actually meant to be played in person with physical components like the board and the cards. And the physical components are actually what distinguish table top games from videogames. This means that an assistant integrated into a "digital version of dixit" would not have worked, because having a mobile app deviates from the whole point of playing a table top game. Actually, a mobile app would not even require the kids to be physically together.

Therefore the main value of seats for all is that we attempted to bring an assistive tool into the "physical and material" reality of table top games. In other words, we tried to bring assistance into the context in which table top games are actually played and in which they are actually meaningful and can help ASD kids to feel accepted. This has been possible (considering the limitations implied by this approach) thanks to the XR environment, the LLM and the detection models for card scanning.

6.2 Why it is a good solution

The previous section expressed our thoughts on why Seats For All is a unique and valuable project. This section, instead, deals with whether the solution implemented is an optimal solution for the hard problem that the system is trying to solve: the problem of supporting the user throughout a game session performed with physical table top game components. The requirements that an hypothetical system must have to solve such problem are the following:

- It should allow the user to see their surrounding
- Suggestions should be only visible or hearable by the user
- The system should have cameras to scan the cards

- The system should have microphones to hear the users.
- The system should be usable without using hands since they are busy holding the cards.
- The system should be able to connect to the internet.
- The system should be movable and easy to set up.

Given these premises, the use of a MR device with passthrough enabled naturally comes as the most obvious solution. This is justified by the fact that most modern VR devices meet all those requirements and hardly any other possible and cheaper setup can. Therefore, a VR device is the only way to meet the requirements imposed by the context in which the system should be used and at the same time guarantee good usability.

Moving on to the implementation and the game itself, the first and biggest problem that the system should solve is helping the child in coming up with a word (in case they are the narrator) or associate a card (in case they are guessers). The reason why this is the major problem is that the choice of the card or the word is the most pressuring phase of the game: the child has relatively little time to make a decision under social pressure and such pressure is amplified by their condition. Even though interactions with Seats still require some time, the child is certain that at the end they will certainly reach a decision. This ensures that they will feel more secure and it will relieve the pressure from them. This is why we believe that the card scanning system is the most relevant and important piece of work that needs to be implemented to reach the system goals.

A second problem is the game rules, remembering all the rules and making sure that a certain move is allowed can also be a source of stress. This is important especially in the points computation phase since the point assignment mechanism contains the most complex rules of Dixit. As discussed in the section about the state of the art, LLMs behave very well as a base knowledge for game rules and they can assist the user in remembering and applying those rules. Therefore the use of an LLM was the most effective way to help the user for such problems.

6.3 Competitors and comparison

Seats for all is a very unique project and has no direct competitors that have approached the problems discussed in section 6.1 in a similar way. However, there are other research projects or commercial products that have tried to assist children with neurodivergencies by helping them to play games. Here follows a list of such competitors each accompanied by the main differences with our project.

Name	Description	Difference with Seats for All
Floreo [23]	VR application offering simulation of social interactions	Not specifically designed for playing a table top game.
Inclusive Imagination [24]	Foundation that aims at developing inclusive table top games for neurodivergent children	Not specifically designed to help children play a commercial game like Dixit
Tovertafel [25]	AR system installed on the ceiling able to project games onto surfaces to help people with cognitive disabilities	Does not specifically target children, hard to set up.
Tabletop Simulator [40]	A multiplayer physics-based sandbox video game that simulates the experience of playing board games in a virtual environment	Is an entirely digital experience, meaning the face-to-face communication element is entirely lacking. Also requires programming the entire game as a 'digital' version

Figure 45: Tabletop simulator logo

7 Future Work

7.1 Critical Reflection

The hardest challenge to solve in this project, as anticipated in section 6, is making the system aware of the game status. What we tried to do to solve this issue is asking the child to explicitly tell this information to the assistant. In fact there are not many actions the system can perform to empirically predict the state of the game. For instance, how can the assistant know which card did the child decide to play? We tried to solve this problem by prompting the assistant to explicitly ask the child to let the system scan the card that they were willing to play. However this solution may well be slow, this is not a problem if the child is a Guesser since every other guesser is thinking about which card to play at the very same moment, but this may be a problem when the child is the narrator and everyone is waiting for the word.

Another problem is that we might have decided to rely more on the visual part other than just the auditory part. For now the only impactful elements of the visual part are the 3D model of the assistant and the word suggestions placed near the card. Certainly, other useful visual elements could have been implemented, an example may be a graphical interface providing information on the game status. This would have helped the child to instantly see the game status without asking Seats. The team was perfectly aware of the potential of the visual components and the reason why such features were not designed and implemented is a general lack of expertise and time.

One last problem is the behavior of the LLM, the system relies a lot on the LLM since every possible help the system can give requires the intervention of the LLM. This of course may lead to problems since LLM, although able to understand and apply the rules perfectly, are still unpredictable. This risk can be mitigated by making the application less reliant on the assistant. Of course part of this problem emerged from the previous challenge of making the system aware of the game status that was discussed a few paragraphs ago. In fact, given the fact that the child has to tell the assistant most of the information on the game status, the assistant is still the main gate between what is happening in the game and what the system sees. Thus, addressing the first issue discussed will also contribute in addressing the last.

7.2 Future developments

Given the critical reflection of the previous section, future developments should target those vulnerabilities spotted in the previous section.

For what concerns the "system awareness" problem, a possible mitigation of this problem can be the improvement of the detection models. In fact, this is the only mechanism that the system can use to retrieve information on the game status that does not rely on the child. To make this system faster and reliable a classification model can replace the detection model. In other words, by using a classification model the system is not just able to "see" a card but also to "recognise" the card as one of the Dixit cards and to uniquely distinguish that card from the others. More formally, this problem requires the creation of 80 classes, one for each Dixit card, and telling which of these classes the user's card belongs to. This allows a much easier implementation of many application components, for instance, a second GTP4o model for card description is not required since every possible card can be associated to a description after classification. However, the most impactful improvement that could be made to the detection system is being able to tell whether a card is held in hand or lying on the table. The reason for this assertion is that if the card is in hand it means that the card belongs to the user deck; instead, if the card lies on the table it means that the card has been played by other players during the narrator card guessing phase. This can allow the system to understand which card the user played by recognizing one of the cards in the user's hand on the table. This will not require the assistant to explicitly ask for this piece of information.

One additional possibility is to train another model that detects the voting tokens. This would allow the system to be able to count how many people voted for the child's card and to automatically compute the score. Although seeming promising, this behavior is very hard to implement since it would require calculating the position of the token in respect to the card in order to deduce that the token is actually voting for that card. The team was meaning to train such models, but plans were quickly discarded after reflecting on such problems.
One final note that can be written on detection models is the fact that their performances should be improved. VR devices still have limited computational capacity and detection models may cause lag. We solved this issue by not letting the detection models to be always on but this, again, prevents the system from knowing the state of the game, therefore other solutions can still be considered.

Another big area for improvement is, as anticipated in the previous section, the visual part. A user interface containing information on the system status is the first and easier visual element that can be implemented. Other ideas are the following:

- visual elements that tell the child how to move their rabbit ahead on the board given their score on the latest round. This is, again, very hard since it requires detection of board cells and the rabbit.

- Word suggestions attached to more than one card at once. This is possible only with a stronger detection model, but being able to compare more possibilities at the same time is very useful for the narrator
- An icon attached to each card indicating the level of affinity between the card and the narrator word. This can enable the guesser to compare more cards at the same time, but, again, it requires stronger detection systems.

As a final remark about visual elements it is imperative to design such elements according to design principles highlighted in this document, for instance, minimalism to avoid sensorial overstimuli.

Finally, other further developments for the system outside the vulnerabilities found in section 7.1 can be:

- Possibility to customize the level of assistance of the system.
- Possibility to customize other aspects of the assistant behavior, for example one may ask the assistant to only give suggestions during the narrator card guessing.
- Possibility to introduce some feedback from the assistant.

In conclusion, this project is ambitious and has been very challenging but it has great potential. The general approach taken for this project may be replicated for other games helping more children with ASD in having fun and finally enjoying more activities with peers.

8 Business Perspectives

Nowadays game designers care a lot about accessibility and inclusivity, unfortunately this often requires designing a more accessible version for their games [24]. However, the real problem is that even when an accessible version of the game exists it is still hard for an atypical child to find people to play with, even more so if these people are neurotypical.

The solution adopted by Seats for All, discussed in chapter 6, can potentially solve this problem by allowing atypical children to play the "standard" version of popular games. In fact, Seats for All itself aims at making Dixit more accessible without modifying the game itself (the only small simplification we have done was to limit the clue to a single word, but, as discussed previously, this was mainly a simplification done for design purposes and the LLM can still support more complex clues). Even though the entire project was centered around Dixit, the solution discussed in chapter 6 can still be applied for other games. Dixit, after all, was chosen because it appeared in the literature (see section 3.4), not because it is the only game to which this solution can be applied.

Therefore, potential extensions of Seats for All for different games may help game designers to make their games more accessible. Of course, since table top games may have very different mechanics, supporting another game would require actually changing the entire application. More specifically, the components of the application that change the most are:

- Detection models: different computer vision models are required since different games feature different physical elements that need to be "seen" by the system.
- LLM prompting: certainly, the LLM needs to be prompted differently to support different rules. Moreover the LLM has to support different function tools.
- System internal logic: the internal state machine of the system needs to be changed to support a new state machine for the new game.
- Graphical elements: some graphical elements like the GUI should be adapted to the new game.

Basically, Seats for All can be turned into a startup that aims to build VR applications to make popular table top games accessible for people with special needs without requiring game designers to modify the game. Consequently, table top game designers will be the customer and the final user will be atypical children.

Of course, this model will not support any possible game, there may be very complex games that require the system to be too complex. In addition to that, as discussed in chapter 7 about

limitations, the final implementation of such systems is indeed very hard and complex even for non-complicated games like Dixit. Nevertheless, modern popular table top games are also designed to be simple so that their rules can be quickly learnt. Therefore the amount of games that Seats for All cannot certainly support will not probably be so relevant. Moreover, in light of the future work presented in chapter 7, even though the final implementation for Seats for All was hard, there is still opportunity for improvement. This means that potentially implementation may be easier in the future after applying better methods or tools for implementation.

One last observation that can be done is that the market of MR tools like visors or wearable devices like smart glasses is increasing. Also detection models are getting better every year. This will probably mean that more powerful visors will be available in the future and they will be cheaper. This will ease the implementation of such systems as more computational power is available and new libraries and optimization tools will be developed. Finally, more people will have interest in such devices and more people will possess one.

References

[1] J. Zhao et al., "Hand-in-Hand: A Communication-Enhancement Collaborative Virtual Reality System for Promoting Social Interaction in Children with Autism Spectrum Disorders," *PubMed*, 2026.

[2] A. Author *et al.*, "Serious Games for Developing Social Skills in Children and Adolescents with Autism Spectrum Disorder: A Systematic Review," *ResearchGate*, 2026.

[3] *B. Surname, "Eggly: Designing Mobile Augmented Reality Neurofeedback Training Games," 2025.*

[4] C. Surname *et al.*, "Bridging the Gap: Enhancing Gaze-Performance Link in Children with ASD through Dual-Level Visual Guidance in MR-DMT," *arXiv*, 2026.

[5] D. Surname *et al.*, "Evaluating the Acceptability and Usability of a Head-Mounted Augmented Reality Approach for Autistic Children with High Support Needs ('Magic Bubbles')," *Proc. ACM*, 2026.

[6] E. Surname *et al.*, "Foldiverse: Augmenting Paper Folding Physiotherapy for Children with Autism via Family-Centered Mixed Reality Design," *ACM Digital Library*, 2026.

[7] F. Surname *et al*., "Application of Augmented Reality Interventions for Children with Autism Spectrum Disorder: A Systematic Review," *Computers*, vol. 12, no. 10, p. 215, 2023. [Online]. Available: https://www.mdpi.com/2073-431X/12/10/215.

[8] *A. Tinterri, F. Pelizzari, G. Vignoli, F. Palladino, and M. Di Padova, "Towards AI-Assisted Board Game-Based Learning: Assessing LLMs in Game Personalisation," 2026.*

[9] G. Surname *et al.*, "Players and Performance: Opportunities for Social Interaction with Augmented Tabletop Games at Centres for Children with Autism," *Proc. ACM Hum.-Comput. Interact.*, vol. 6, no. ISS, 2022.

[10] D. Thompson, R. Kilmer *et al.*, "Systematic Review: Are Table-Top Role-Playing Games a Useful Therapeutic Tool for People with Neurodiversity?," 2025.

[11] Microsoft Azure, "Azure audio events reference," 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-foundry/openai/realtime-audio-reference. [Accessed: Feb. 12, 2026].

[12] *E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.*

[13] *Discord Inc., "Discord Developer Portal," 2026. [Online]. Available: https://discord.com/developers/docs.*

[14] *J.-L. Roubira, Dixit. Poitiers, France: Libellud, 2008.*

[15] *Roboflow Inc., "Roboflow: Computer Vision Management Platform," 2026. [Online]. Available: https://roboflow.com.*

[16] *Ultralytics Inc., "Ultralytics HUB: The No-code Platform for Training YOLO Models," 2026. [Online]. Available: https://ultralytics.com/hub.*

[17] *Google, "Google Colaboratory," 2026. [Online]. Available: https://colab.research.google.com.*

[18] *G. Jocher, A. Chaurasia, and J. Qiu, "YOLOv8 by Ultralytics," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.*

[19] *ONNX Project, "Open Neural Network Exchange (ONNX) Documentation," 2026. [Online]. Available: https://onnx.ai.*

[20] *G. Atherton, E. Dawson, and L. Cross, "Metaphors and myths: Using the board game Dixit to understand the autistic lived experience," Discover Psychology, vol. 5, no. 1, p. 10, Jan. 2025. doi: 10.1007/s44202-025-00340-9.*

[21] *L. Cross, G. Atherton, S. J. Wilson, and J. P. Taylor, "Game Changer: Exploring the Role of Board Games in the Lives of Autistic People," Journal of Autism and Developmental Disorders, 2024. doi: 10.1007/s10803-024-06408-0.*

[22] Jelle J. Sijtsema and Siegwart M. Lindenberg, “Peer influence in the development of adolescent antisocial behavior: Advances from dynamic social network studies,” *Developmental Review*, vol. 50, pp. 140–154, Dec. 2018, doi: 10.1016/j.dr.2018.08.002.

[23] Floreo Inc., "Floreo: Science-based VR for neurodiversity," 2026. [Online]. Available: https://www.floreovr.com.

[24] Inclusive Imagination, "Inclusive Imagination: Empowering neurodiversity through creative technology," 2026. [Online]. Available: https://inclusiveimagination.com.

[25] Tover, "Tovertafel: The original magic table for inclusive play," 2026. [Online]. Available: https://www.tover.care.

[26] Unity Technologies, "Unity Game Engine," version 2022.3 LTS, 2023. [Online]. Available: https://unity.com.

[27] GitHub, Inc., "GitHub: Where the world builds software," 2026. [Online]. Available: https://github.com.

[28] Meta Platforms Technologies, "Meta MR Utility Kit," version 81.0.0, 2026. [Online]. Available: https://assetstore.unity.com/packages/tools/integration/meta-xr-utilities-sdk-261898.

[29] Meta Platforms Technologies, "Meta XR Core SDK," version 81.0.0, 2026. [Online]. Available: https://developers.meta.com/horizon/downloads/package/meta-xr-core-sdk.

[30] Meta Platforms Technologies, "Meta XR Interaction SDK," version 81.0.0, 2026. [Online]. Available: https://developers.meta.com/horizon/downloads/package/meta-xr-interaction-sdk-ovr-samples.

[31] Meta Platforms Technologies, "Meta XR Interaction SDK Essentials," version 81.0.0, 2026. [Online]. Available: https://developers.meta.com/horizon/documentation/unity/unity-package-manager.

[32] Meta Platforms Technologies, "Meta XR Simulator," version 81.0.0, 2026. [Online]. Available: https://assetstore.unity.com/packages/tools/integration/meta-xr-simulator-261900.

[33] Unity Technologies, "Unity Sentis: On-device AI inference engine," version 2.1.0, 2026. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.sentis@latest/manual/index.html.

[34] Apple Inc., “Siri,” Apple. [Online]. Available: https://www.apple.com/siri/

[35] Blender Foundation, “Blender,” *Blender.org*. [Online]. Available: https://www.blender.org/

[36] Unity Technologies, “OpenXR Plugin (com.unity.xr.openxr),” *Unity Manual (Unity 6.0+)*. [Online]. Available: https://docs.unity3d.com/6000.3/Documentation/Manual/com.unity.xr.openxr.html

[37] Unity Technologies, “Unity OpenXR Meta (com.unity.xr.meta-openxr),” *Unity Manual (Unity 6.0+)*. [Online]. Available: https://docs.unity3d.com/6000.3/Documentation/Manual/com.unity.xr.meta-openxr.html

[38] Unity Technologies, “Unity Asset Store,” *Unity Asset Store*. [Online]. Available: https://assetstore.unity.com/

[39] C. Nolet, “Quick Outline,” *Unity Asset Store*. [Online]. Available: https://assetstore.unity.com/packages/tools/particles-effects/quick-outline-115488

[40] Berserk Games, *Tabletop Simulator*, computer software. [Online]. Available: https://store.steampowered.com/app/286160/Tabletop_Simulator/